



# NXP's Model-Based Design Toolbox for S32Z/E: Hands-on Session

# Agenda Overview

- Introduction to S32Z2/E2 Model Based Design Toolbox (MBDT)
- Exercise 1: LED toggle
- Exercise 2: Monitoring with FreeMASTER
- Exercise 3: Counter in Processor in the Loop
- Exercise 4: Multicore Execution



# Introduction

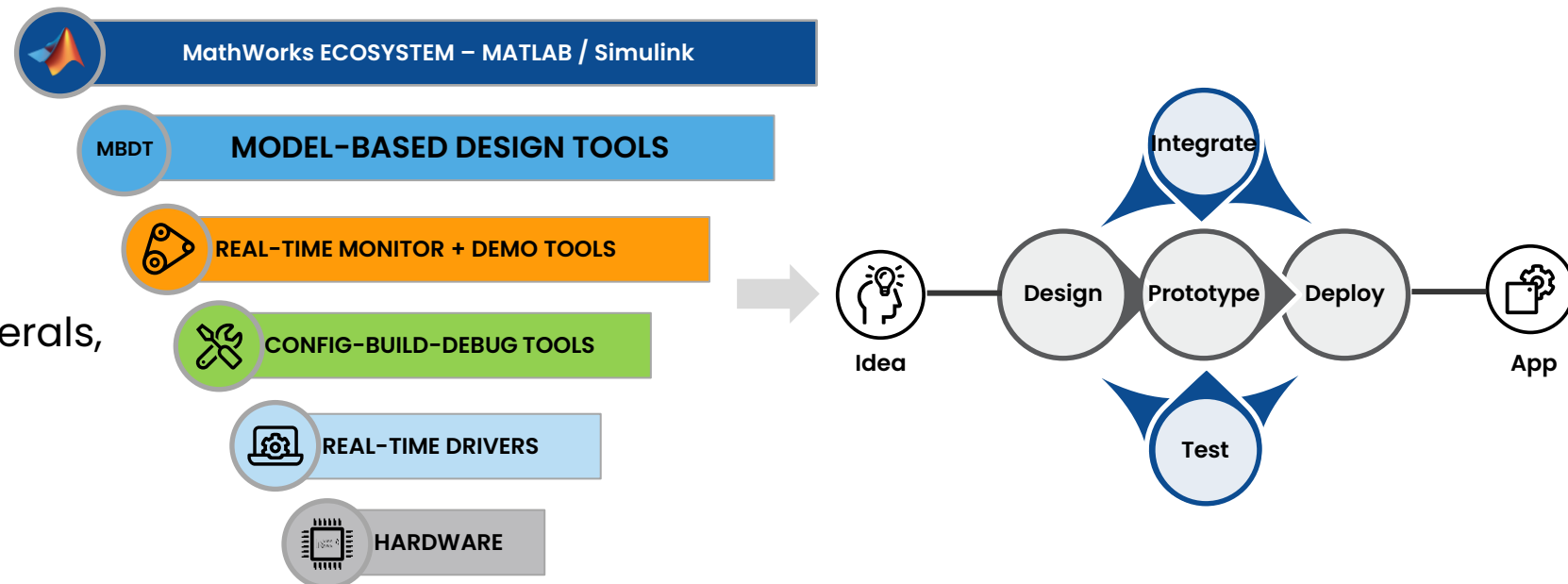
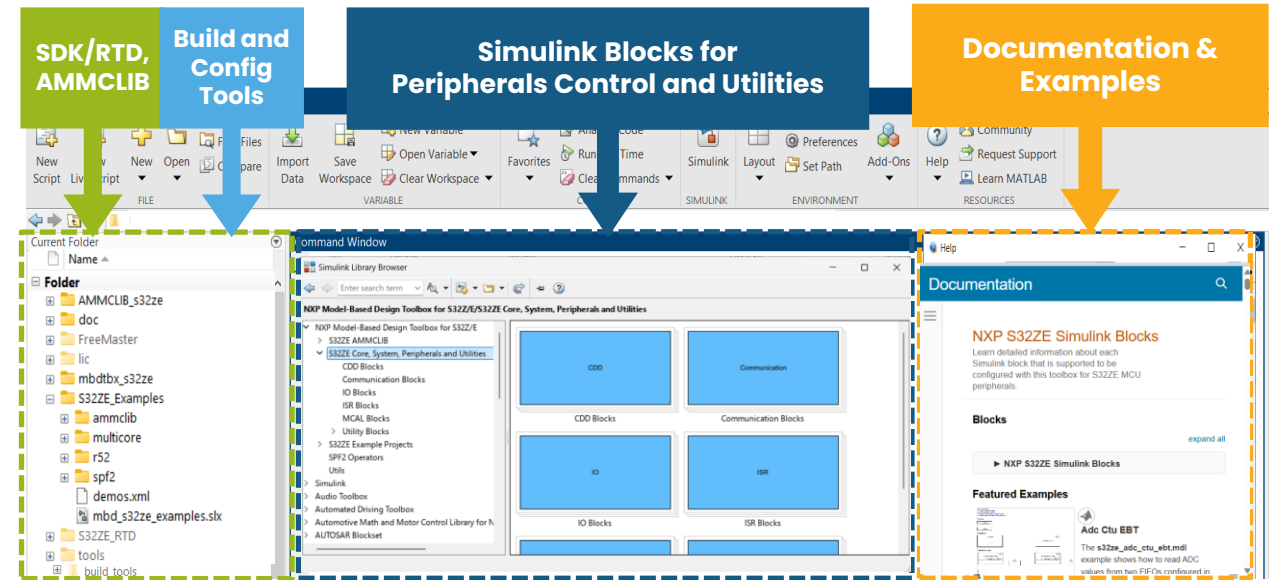
S32Z2/E2 & MBDT



# Model-Based Design Toolbox

*"A collection of Tools & Libraries designed to assist customers with prototyping and accelerate algorithm development on NXP MCUs from MATLAB and Simulink®"*

- ✓ Free-of-charge
- ✓ Automatic Code Generation
- ✓ Build Toolchain
- ✓ Download to NXP Embedded Targets
- ✓ Software-in-the-Loop
- ✓ Processor-in-the-Loop
- ✓ Real-Time Drivers Integration
- ✓ I/O, Communication, Timers, Analog Blocks
- ✓ External Tools integration for Peripherals, Pins, and Clock configuration
- ✓ On-target Profiling
- ✓ NXP FreeMASTER built-in support
- ✓ Ready to run examples

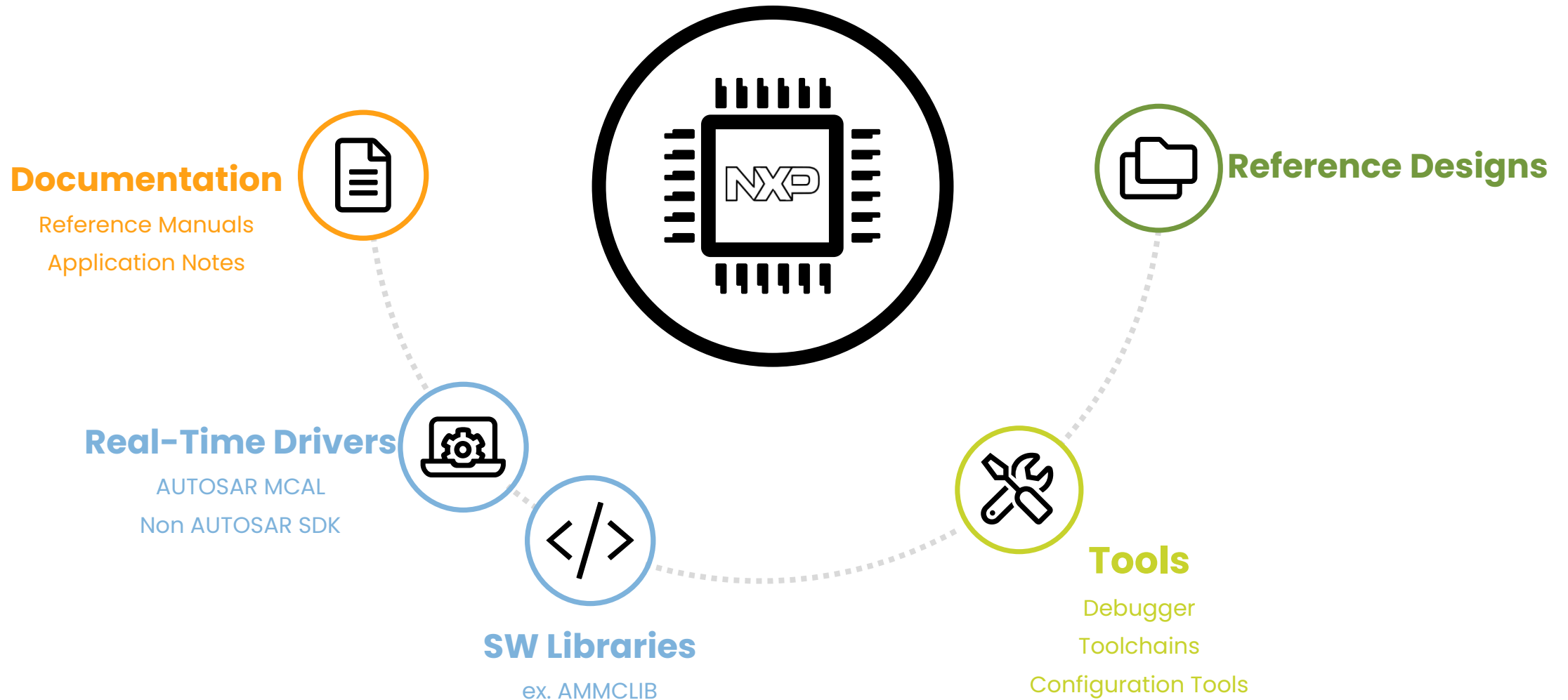


# Hardware Peripherals Configuration

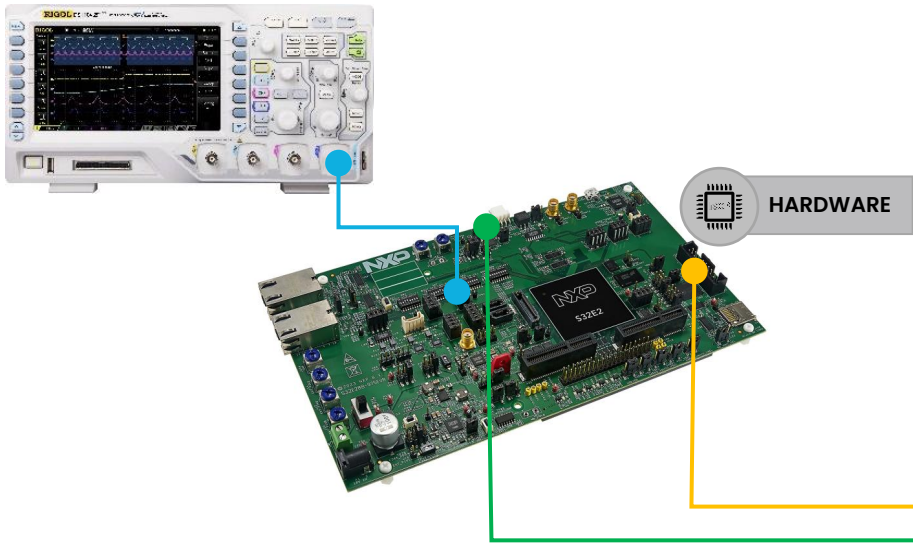
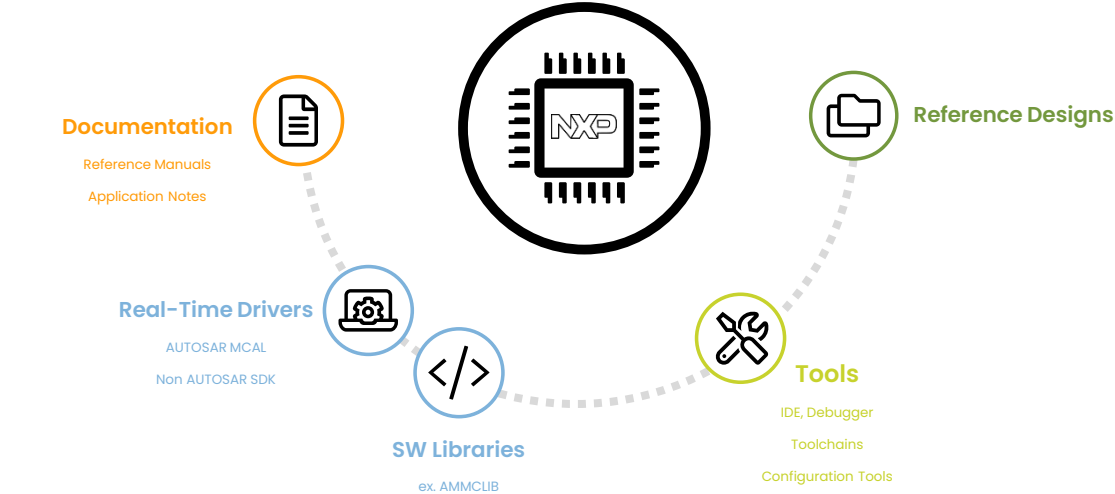
NXP Tools Ecosystem &  
External Configuration Tools



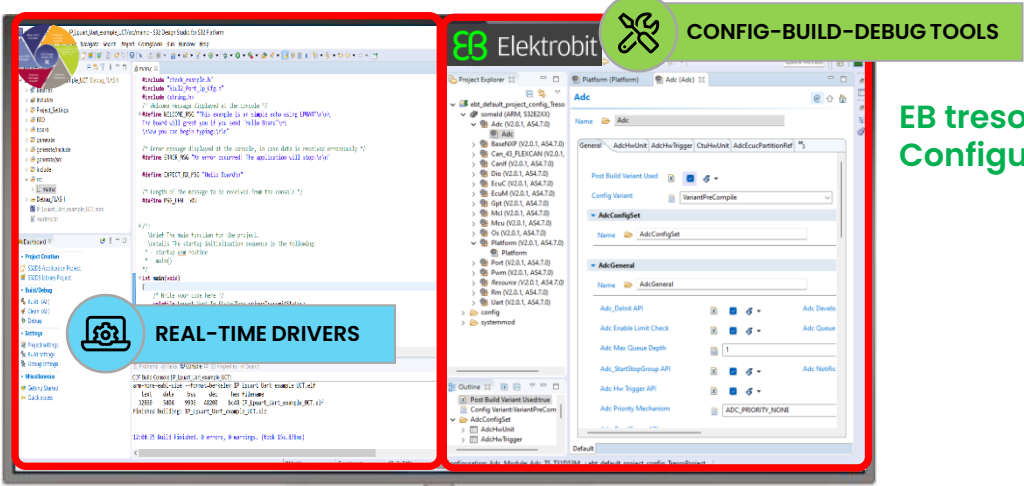
# NXP Software and Tools



# NXP Software and Tools Ecosystem



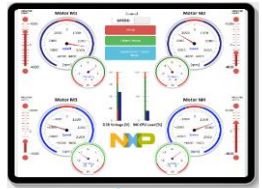
## S32 Design Studio – IDE – Debugger – Toolchains



## EB tresos Configuration Tool

## FreeMASTER

## FreeMASTER Lite



## REAL-TIME MONITOR + DEMO TOOLS



# NXP Software and Tools Ecosystem

 **REAL-TIME MONITOR + DEMO TOOLS**

 **CONFIG-BUILD-DEBUG TOOLS**

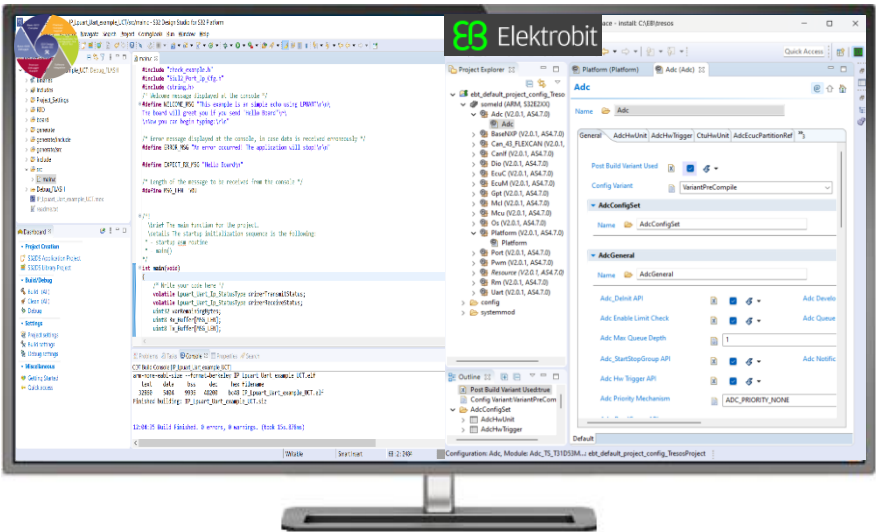
 **REAL-TIME DRIVERS**

 **HARDWARE**



JTAG  
UART

## S32 Design Studio – IDE – Debugger – Toolchains

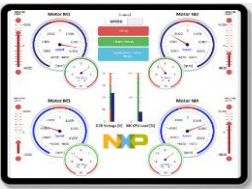


**EB tresos  
Configuration Tool**

FreeMASTER



**FreeMASTER Lite**





# NXP Software and Tools Ecosystem



MathWorks ECOSYSTEM – MATLAB / Simulink



REAL-TIME MONITOR + DEMO TOOLS



CONFIG-BUILD-DEBUG TOOLS

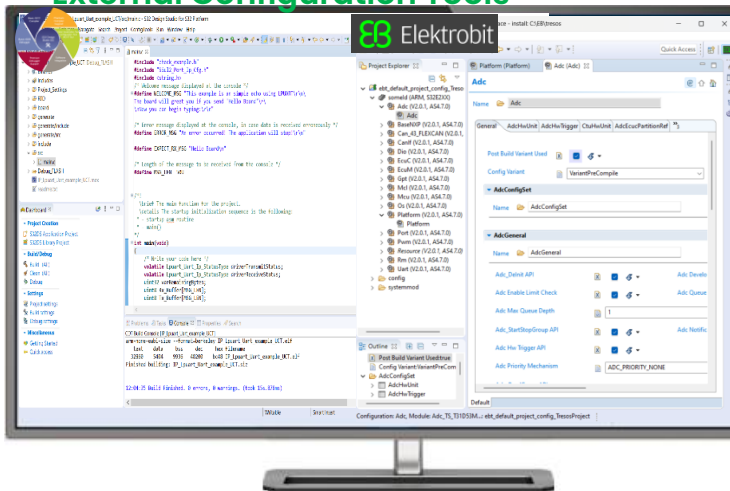


REAL-TIME DRIVERS

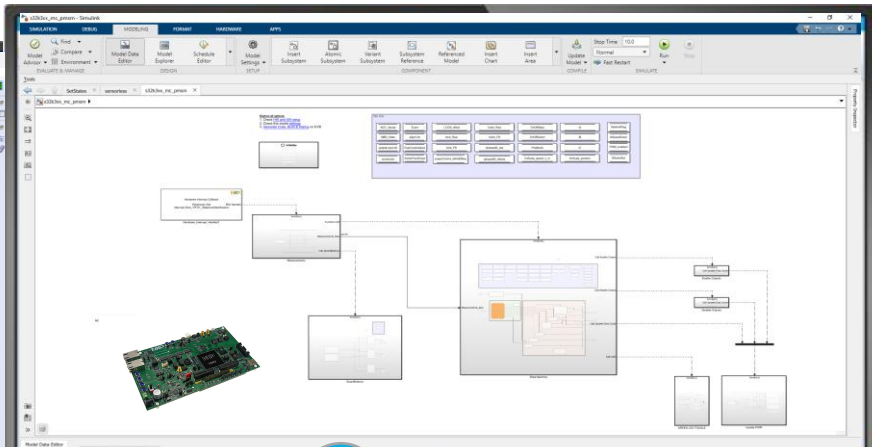


HARDWARE

S32 Design Studio  
IDE – Debugger – Toolchains  
External Configuration Tools

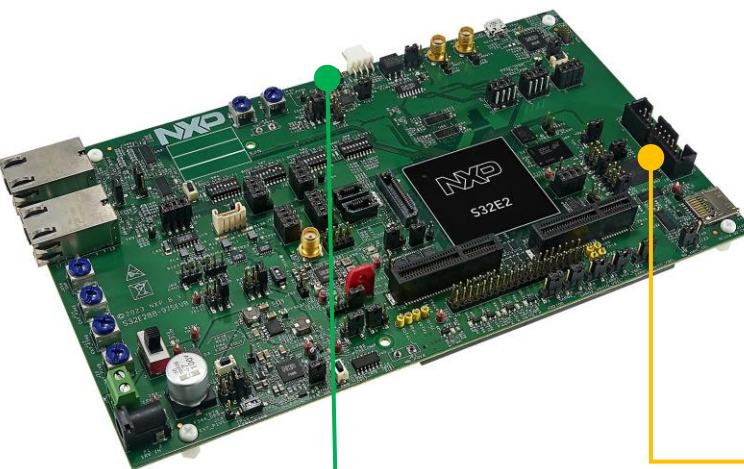


Model-Based Design Tools for Simulink



MBDT

MODEL-BASED DESIGN TOOLS



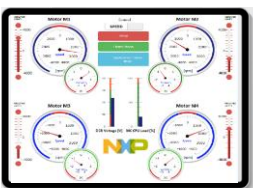
JTAG

UART

FreeMASTER



FreeMASTER Lite



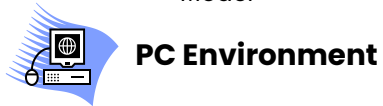
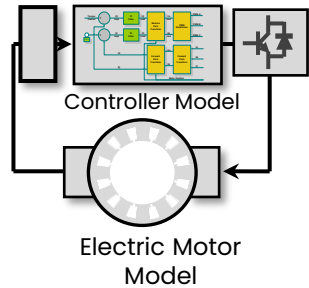
# Simulation Modes

MIL-SIL-PIL-Standalone



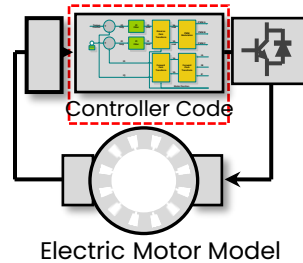
# Model-Based Design Workflow

## Idea incubation



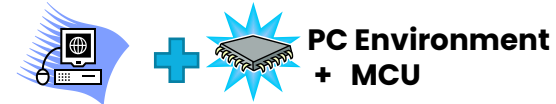
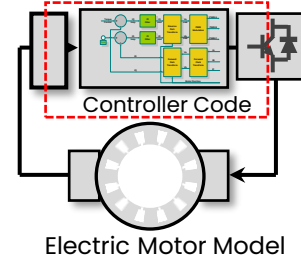
## Automatic Code Generation

To SIL



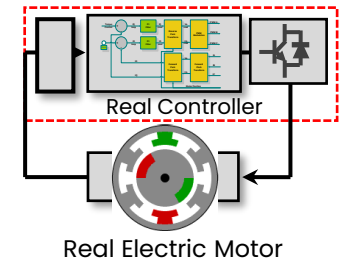
## Code Validation

To PIL



## Final Product

To MCU



### Step 1 – System Requirements: Model-in-the-Loop

- Software requirements
- Control system requirements
- Overall application control strategy

### Step 2 – Modeling/Simulation: Software-in-the-Loop

- Control algorithm design
- Code generation preparation
- Control system design
- Start testing implementation

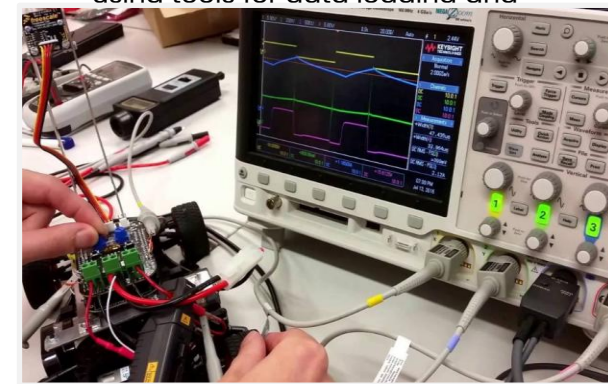
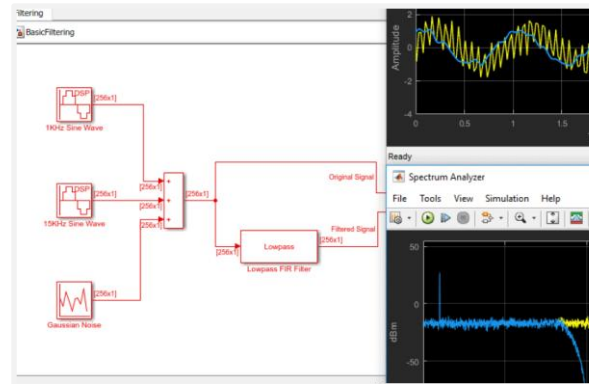
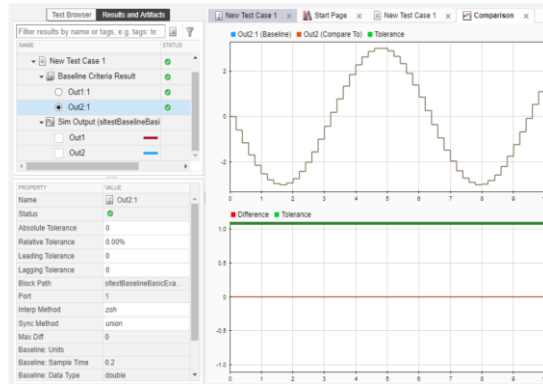
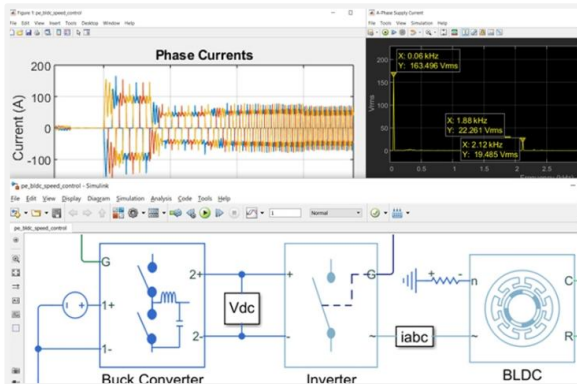
### Step 3 – Rapid Prototype: Processor-in-the-Loop

- Controller code generation
- Determine execution time on MCU
- Verify algorithm on MCU
- See memory/stack usage on MCU

### Step 4 – Target MCU Implementation

#### MCU Final Application

- Validation/verification phase
- Controller code generation
- Test system in target environment using tools for data loading and



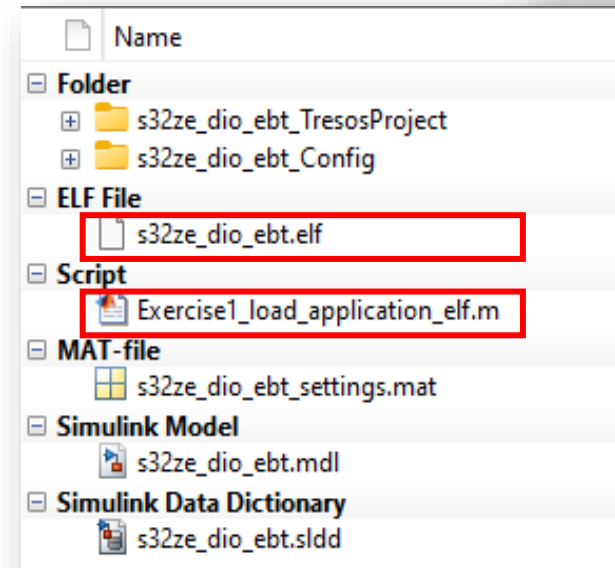
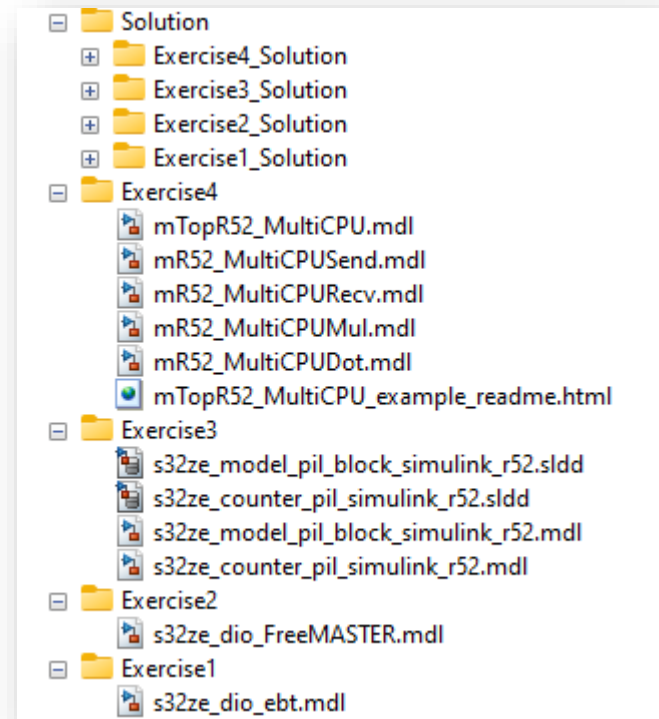
# Hands-On Exercises

DIO toggle



# Overview

- The files for the 4 exercises are stored in the following location:  
"C:/Documents/S32ZE\_DetroitTechDay/HandsOn\_Session"
- To solution to each exercise is provided under the **Solution** folder
- For exercises 1, 2 and 4, a utility script is provided to directly load the executable file to the S32E2 EVB using the debugger



# Exercise 1

DIO toggle





## Exercise 1: Overview

This hands-on exercise introduces the basics of using **NXP's MBDT** with the **S32E288-975EVB**. It serves as a foundational starting point for embedded development using Simulink and MBDT.

### What You Will Learn

- How to configure and toggle a **GPIO pin** using Simulink.
- How to deploy a simple model to the **S32E288 EVB**.
- How to verify hardware interaction by blinking an onboard or external **LED**.

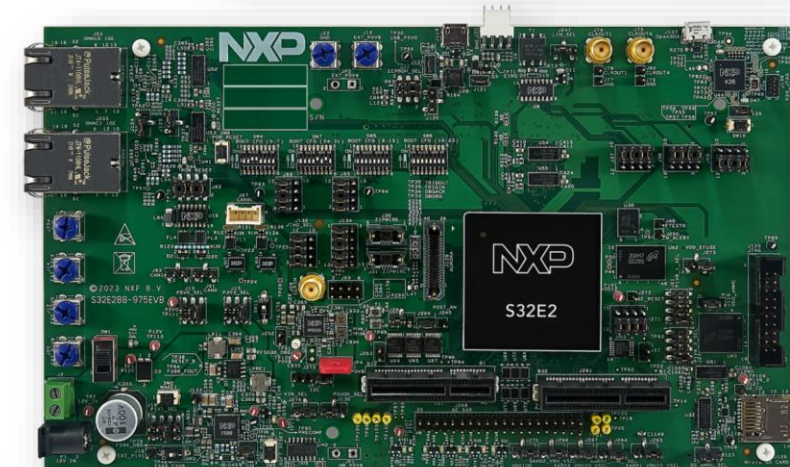
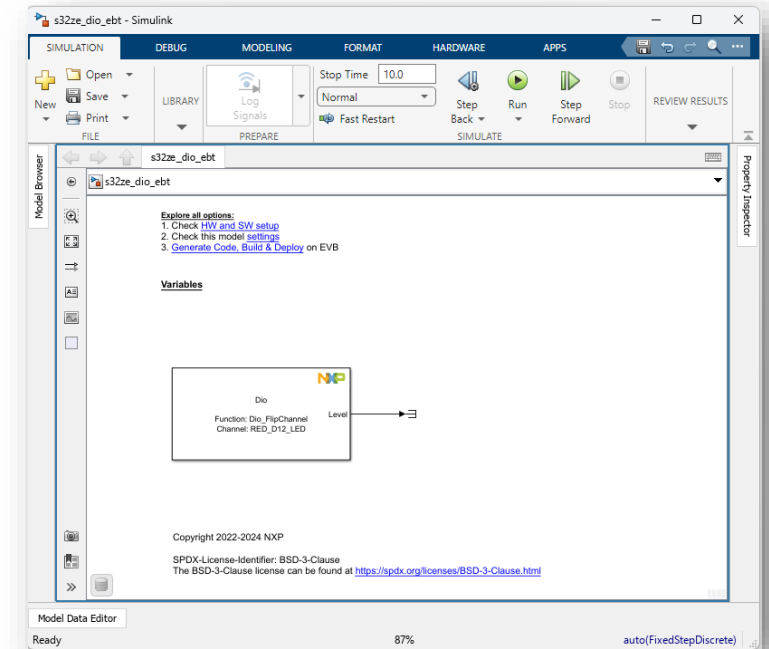
### Expected Outcome

- Successfully build and deploy a Simulink model to the EVB.
- Observe a blinking LED controlled by the model.
- Gain confidence in using MBDT for future, more complex embedded applications.

# Exercise 1: Overview

For this exercise, we will start with a blank Simulink model and configure it to achieve the following:

- Build and generate code for the **S32E288-bga975 processor**
- Use the **NXP MBDT Library** to configure GPIO functionality
- Configure a **Dio** to toggle D12\_LED
- Deploy the project onto the **S32E288-975EVb**
- Successfully blink the D12\_LED on the EVB

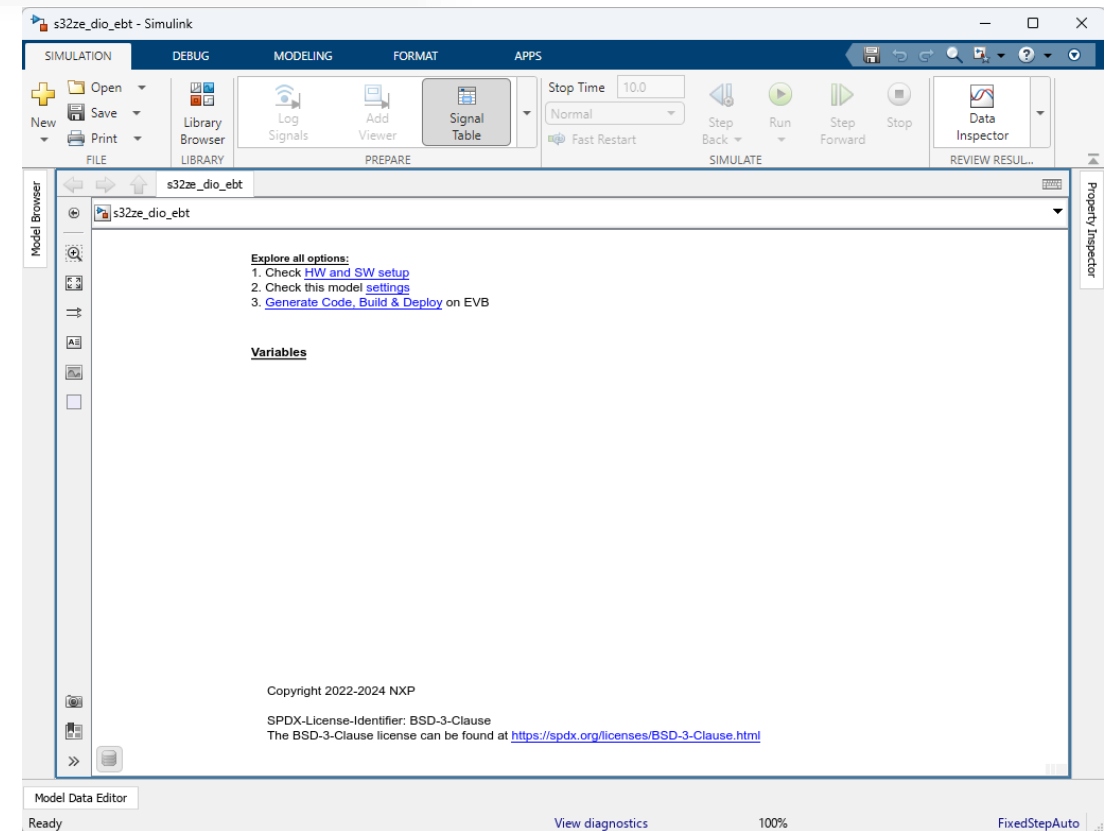
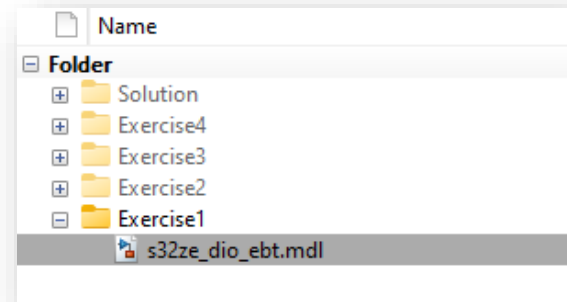


D12\_LED

# Exercise 1: Step 1

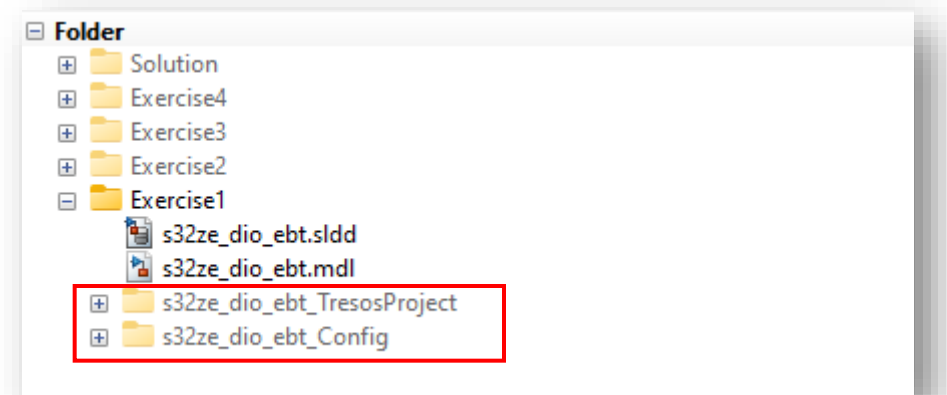
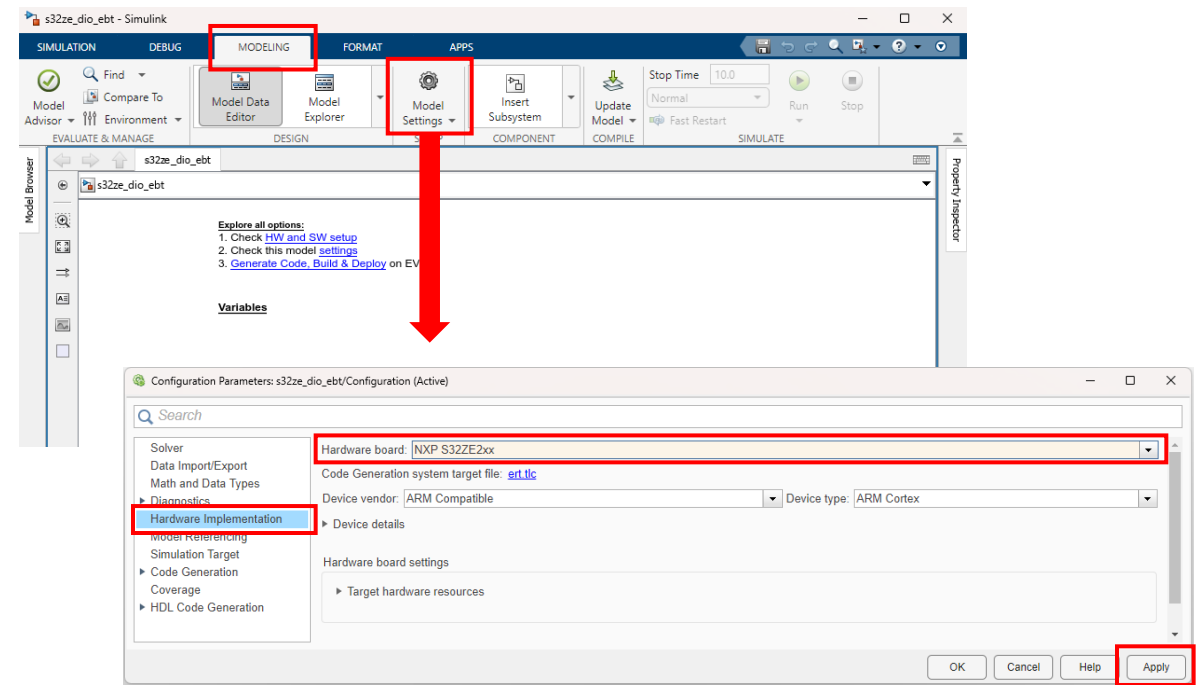
The hands-on session exercises folder contains all the models used for this sessions with the solution models for reference

1. Open the “**s32ze\_dio\_ebt.mdl**” blank model from **Exercise1** folder
2. The blank Simulink model provide us with the canvas to create the logic for toggling the LED



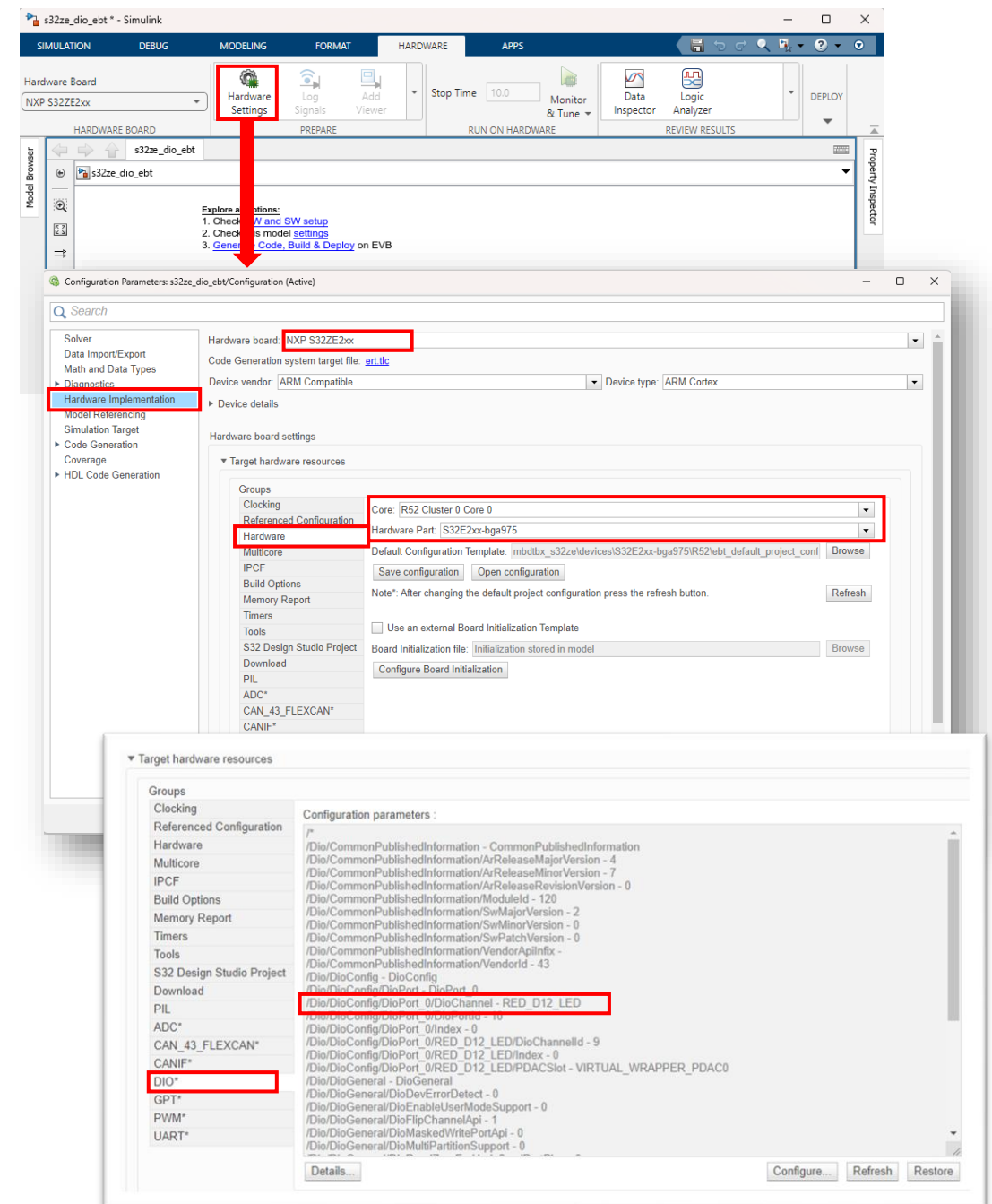
## Exercise 1: Step 2

1. From the MODELING tab, open the **Model Settings**
2. In the **Hardware Implementation** tab of the configuration parameters window, select the **NXP S32ZE2xx** as the hardware board and click on **Apply**
3. A successful configuration of the board will result in auto generating a default **S32 Design Studio** and **EB Tresos** projects



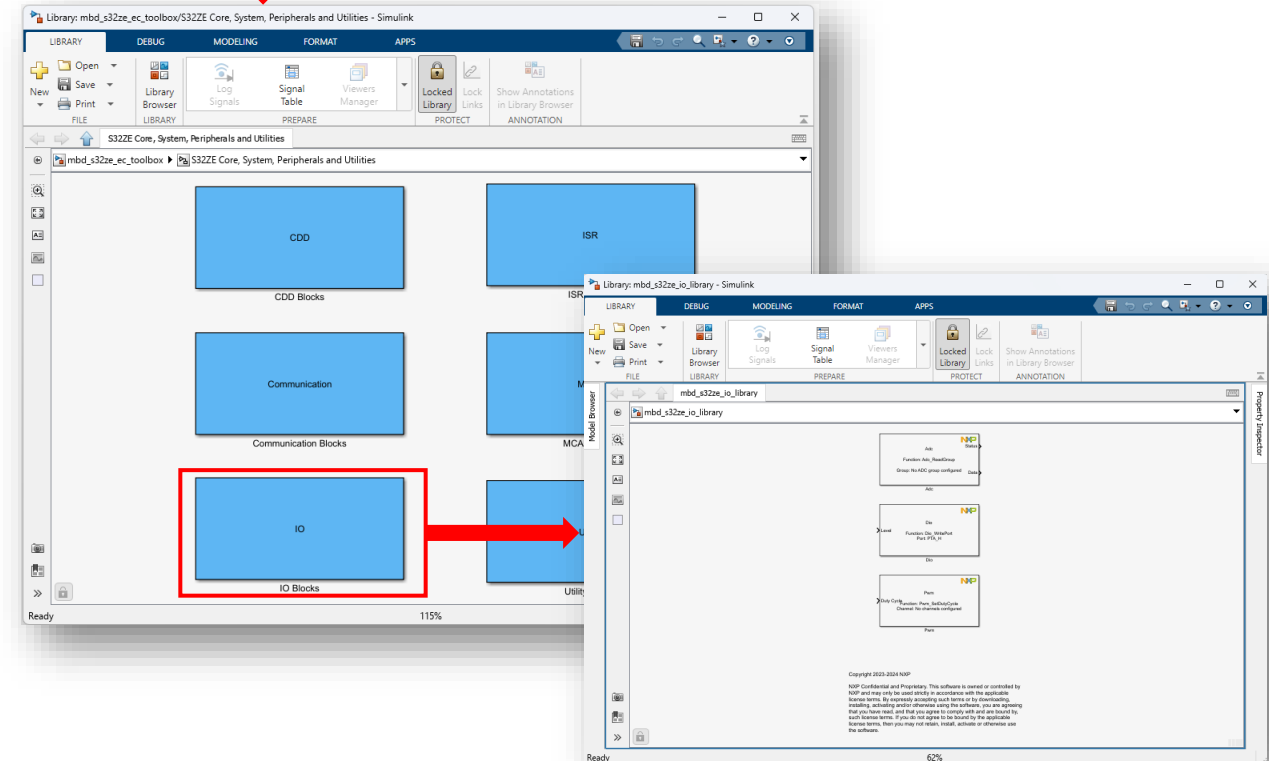
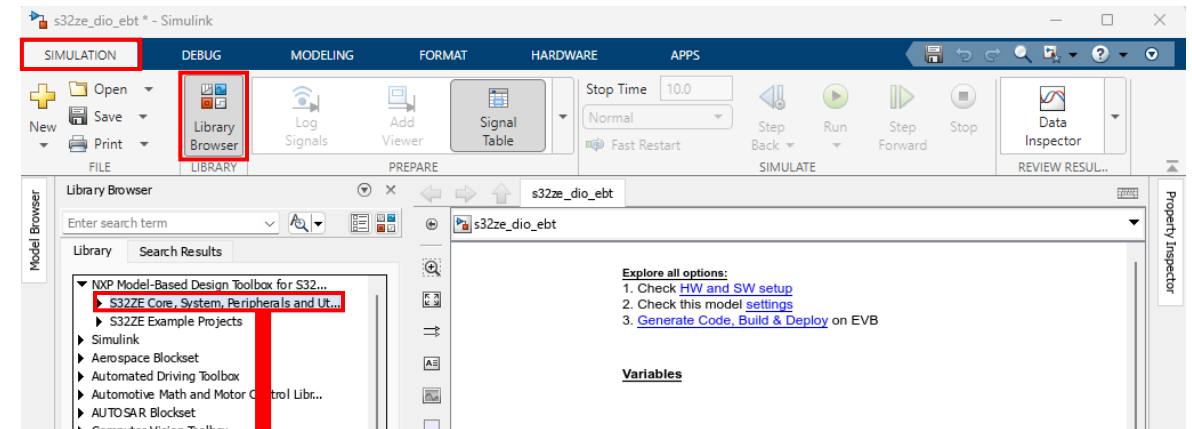
## Exercise 1: Step 3

1. Within the Hardware Implementation, expand the **Target hardware resources**
2. Select the **Hardware** view under the Target hardware resources
3. Ensure the selected core is "**R52 Cluster 0 Core 0**"
4. Ensure the Hardware Part is "**S32E2xx-bga975**"
5. Optional steps:
  - Select **DIO** under the Target hardware resources
  - By default, **D12\_LED** is configured



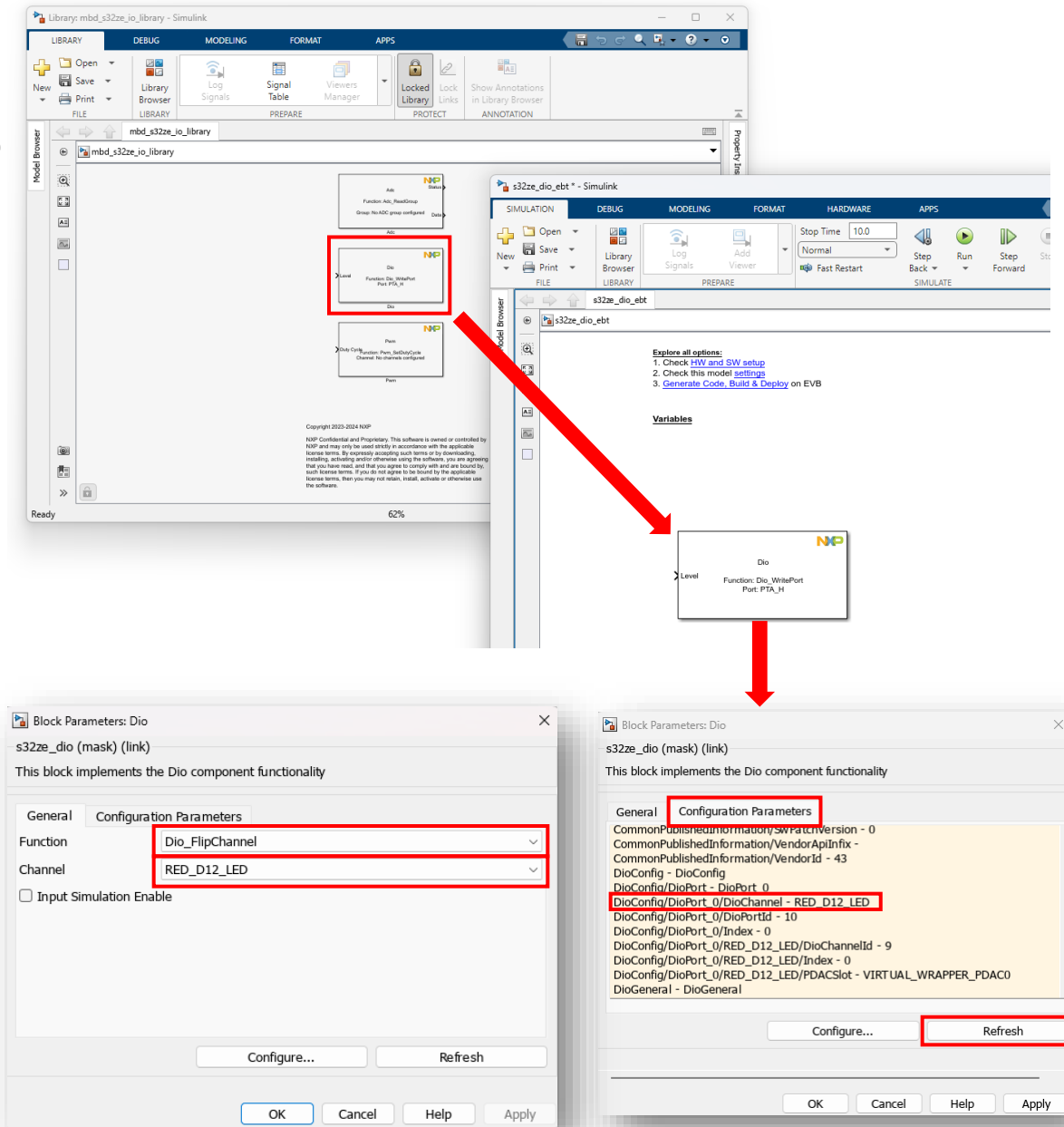
# Exercise 1: Step 4

1. From the SIMULATION tab, open the **Library Browser**
2. Open the **S32ZE Core, System, Peripherals and Utilities** library
3. Open the **IO Blocks** library that includes Dio, Adc, and PWM NXP blocks



## Exercise 1: Step 5

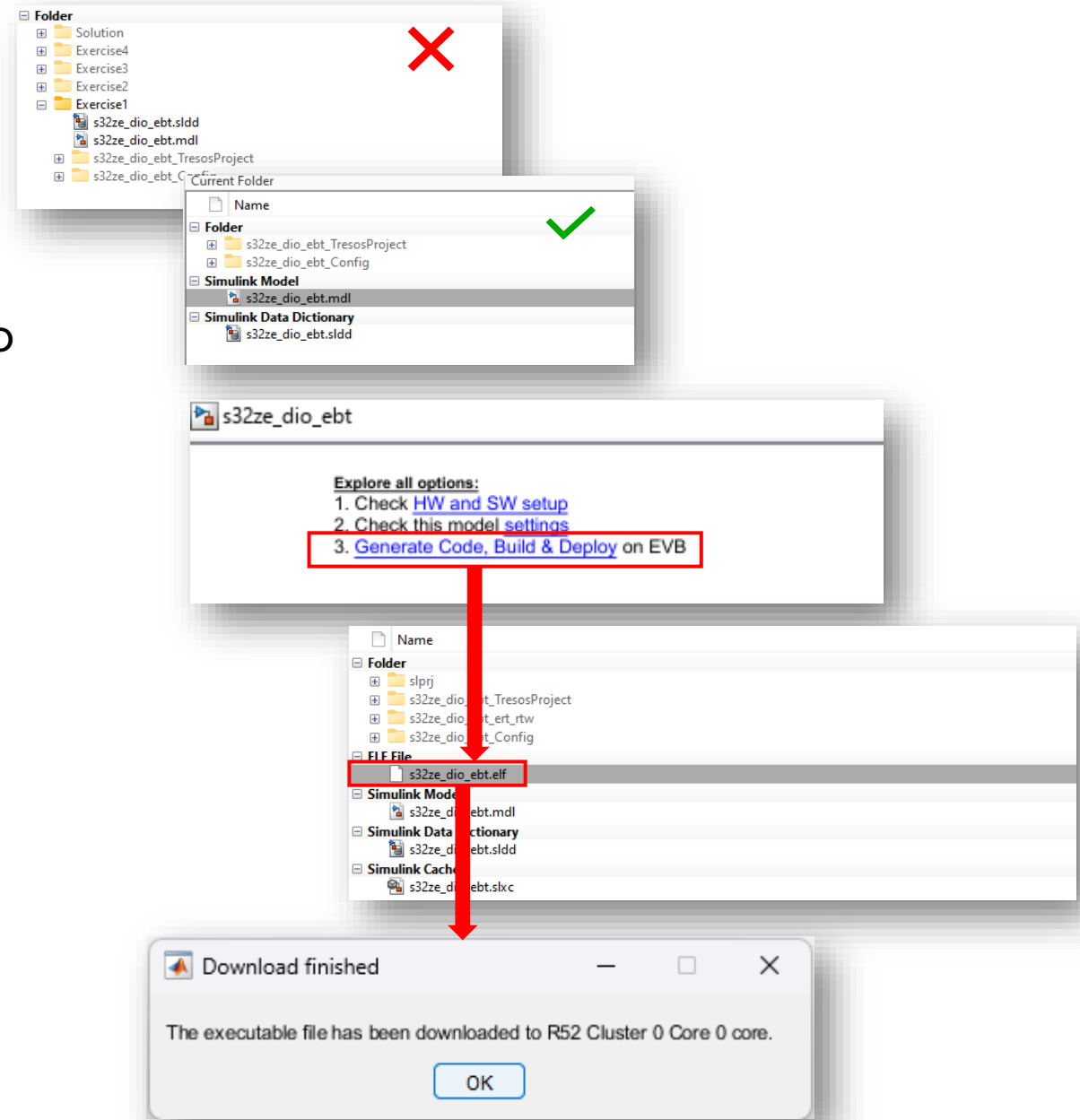
1. From the **IO Blocks** library click and drag the Dio block onto the blank Simulink model
2. Double click on the **Dio block** in the Simulink model to open the block parameters
3. In the **Dio Block Parameters**, click Refresh to ensure the model configuration reflects in the Configuration Parameters
4. In the Dio Block Parameters, under the **General** tab, select the "**Dio\_FlipChannel**" function and the "**RED\_D12\_LED**" channel and disable the Input Simulation Enable





## Exercise 1: Step 6

1. Save the model and ensure that MATLAB's **current folder** is Exercise1
2. Connect the S32 Debugger through JTAG to the **S32E288-975EVB** and power up the board
3. Click on **Generate Code, Build & Deploy**
4. A successful build and deploy will result in the D12\_LED blinking every second



# Exercise 2

## Monitoring with FreeMaster



## Exercise 2: Overview

This hands-on exercise introduces the basics of how to create applications using NXP's MBDT and how to monitor their execution using NXP's **FreeMASTER** tool

### What You Will Learn

- How to configure **FreeMASTER** through NXP's MBDT.
- How to **monitor the execution** of an application with FreeMASTER.
- How to **profile execution** of an application with FreeMASTER.

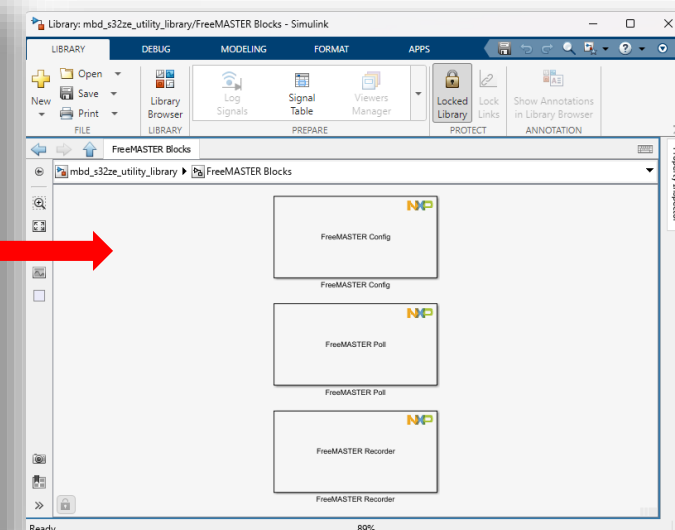
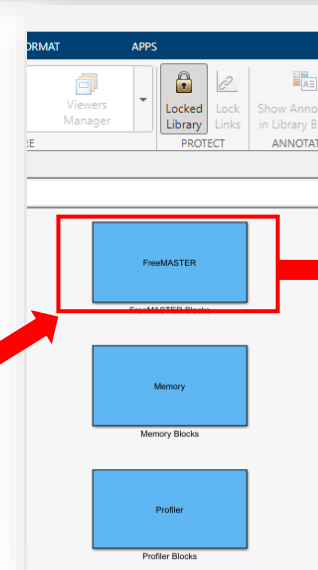
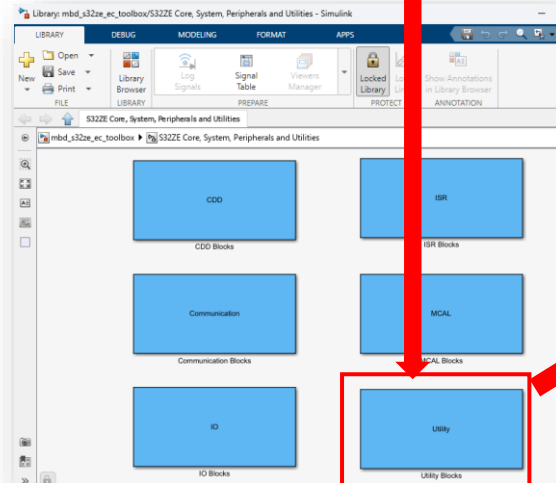
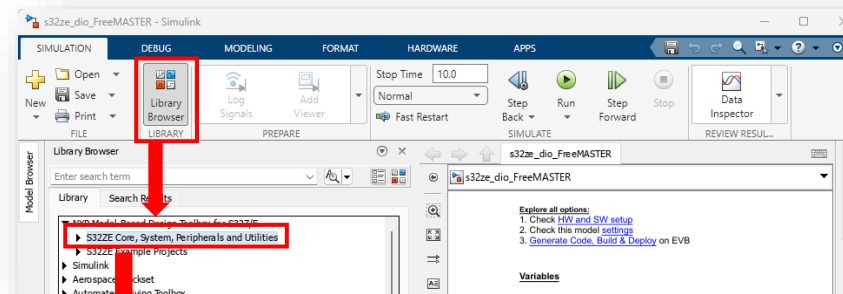
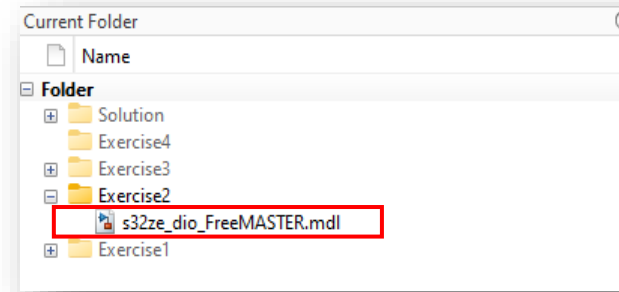
### Expected Outcome

- Successfully build and deploy a Simulink model to the EVB that contains FreeMASTER tooling.
- Monitor the real-time values of variables on the hardware through FreeMASTER.
- Monitor the profiling of the model's execution through FreeMASTER.

## Exercise 2: Step 1

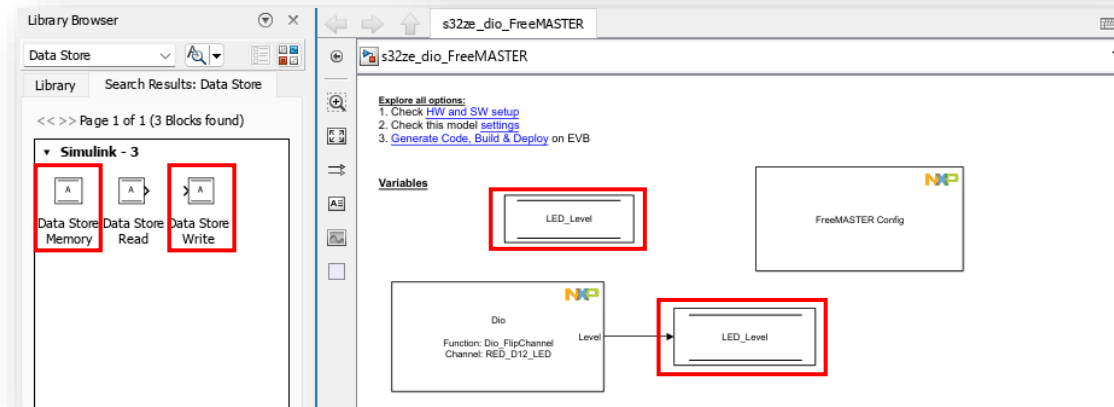
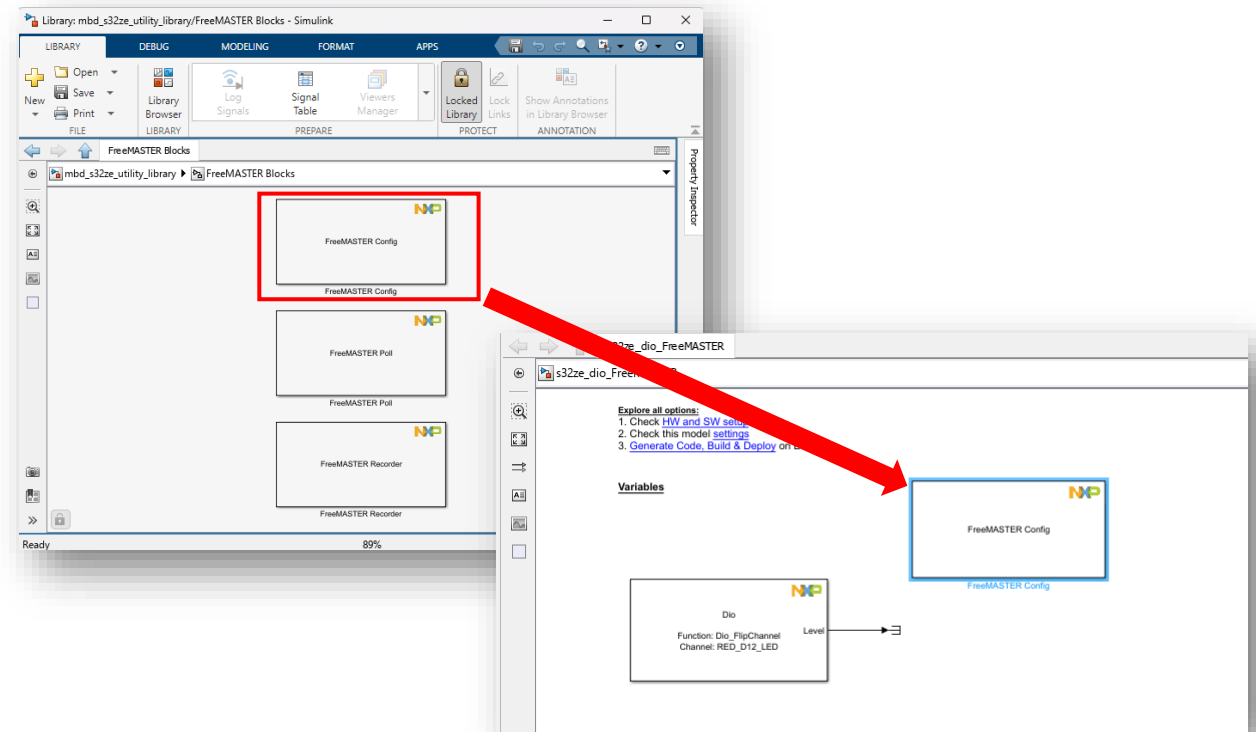
For this exercise we will continue using the LED blinking model.

1. Open the  
    **"S32ze\_dio\_FreeMASTER.mdl"** from  
    folder **"Exercise2"**
2. Open the NXP MBDT library
3. In the MBDT library Navigate to **"Utility  
    block > FreeMASTER Blocks "**
4. Open the **FreeMASTER Blocks**



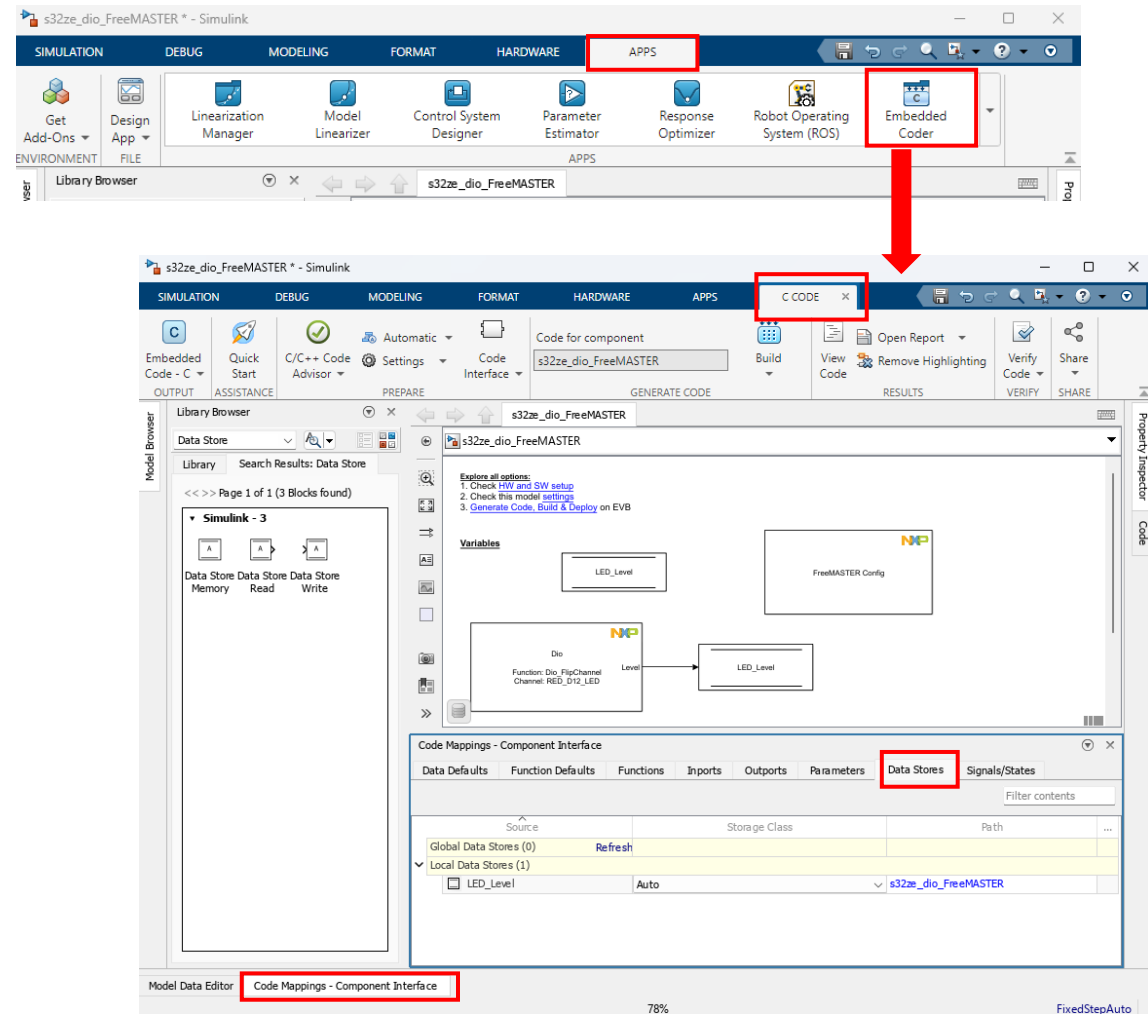
## Exercise 2: Step 2

1. From the FreeMASTER Blocks library, drag and drop the **FreeMASTER Config** block into the model
2. In the library browser search for the **"Data Store"** blocks
3. Drag and drop the **"Data Store Memory"** and the **"Data Store Write"** from the library to the model
4. Connect the output of the Dio block to the Data Store Write
5. Rename the variable to "LED\_Level"



## Exercise 2: Step 3

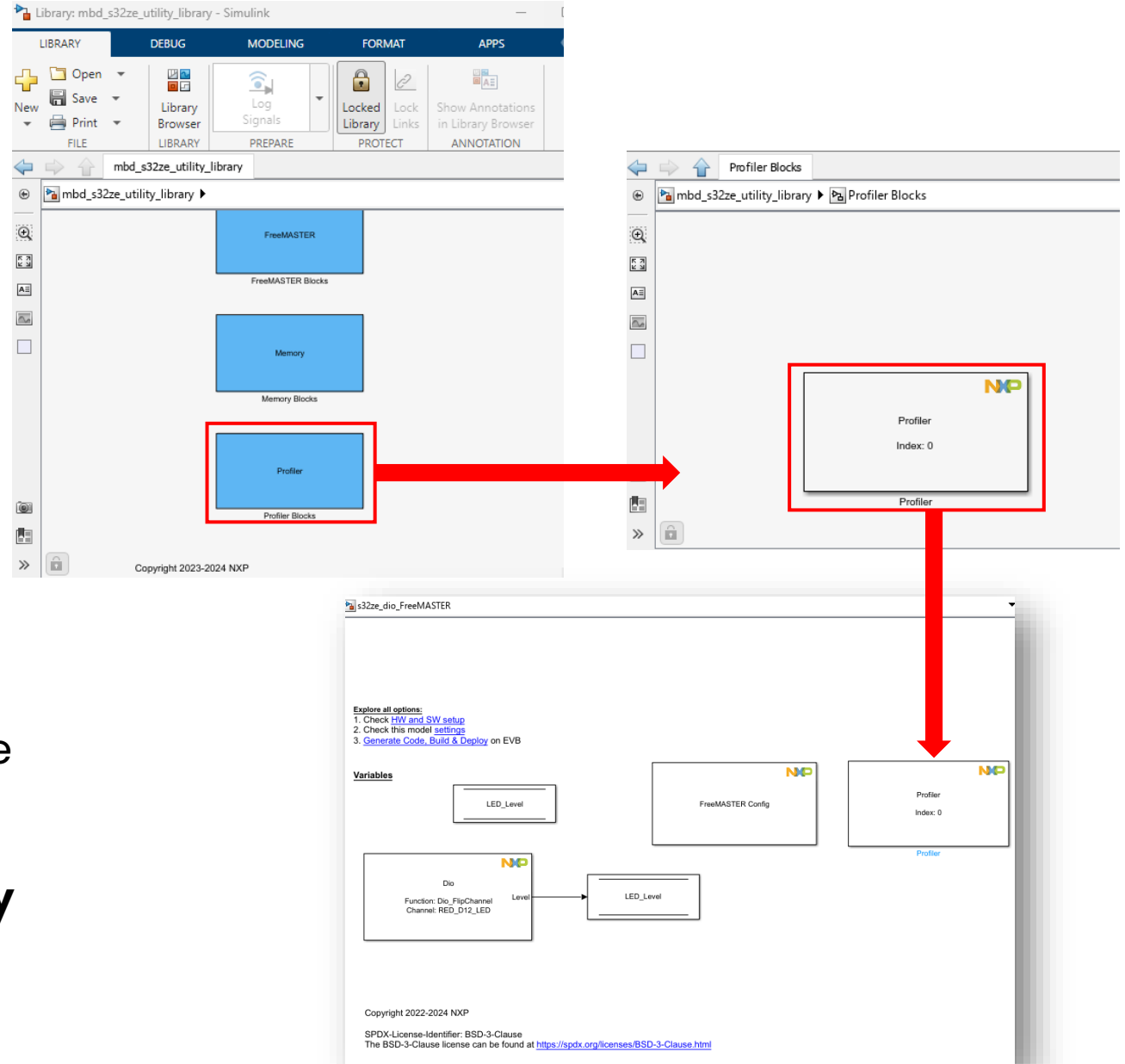
1. From the **APPS** tab open the **Embedded Coder**
2. in the **"C CODE"** tab open the **"Code Mappings – Component Interface"**
3. Select the **"Data Stores"** tab in the Code Mappings interface
4. Change the storage class of LED\_Level to Volatile



| Data Defaults          | Function Defaults | Functions     | Imports | Outputs              | Parameters | Data Stores | Signals/States |
|------------------------|-------------------|---------------|---------|----------------------|------------|-------------|----------------|
| Filter contents        |                   |               |         |                      |            |             |                |
| Source                 |                   | Storage Class |         | Path                 |            |             |                |
| Global Data Stores (0) |                   | Refresh       |         |                      |            |             |                |
| Local Data Stores (1)  |                   |               |         |                      |            |             |                |
| LED_Level              |                   | Volatile      |         | s32ze_dio_FreeMASTER |            |             |                |

## Exercise 2: Step 4

1. From the MBDT **Utility Blocks** open the **Profiler Blocks** library
2. Drag and Drop the **Profiler** block from the library to the model
3. Connect the S32 Debugger through JTAG to the **S32E288-975EVB** and power up the board
4. Connect the EVB to the laptop using the provided microUSB cable
5. Click on **Generate Code, Build & Deploy**

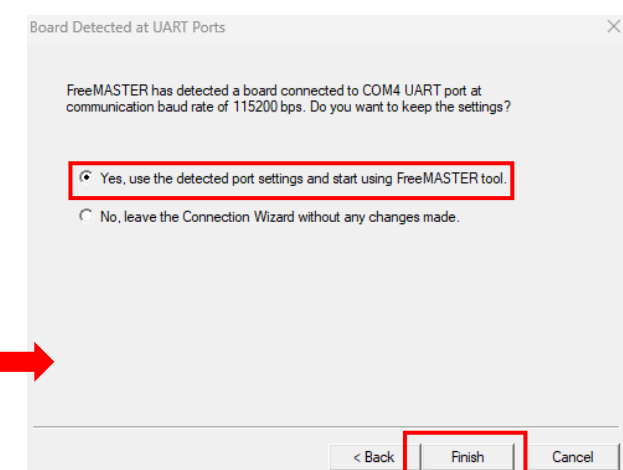
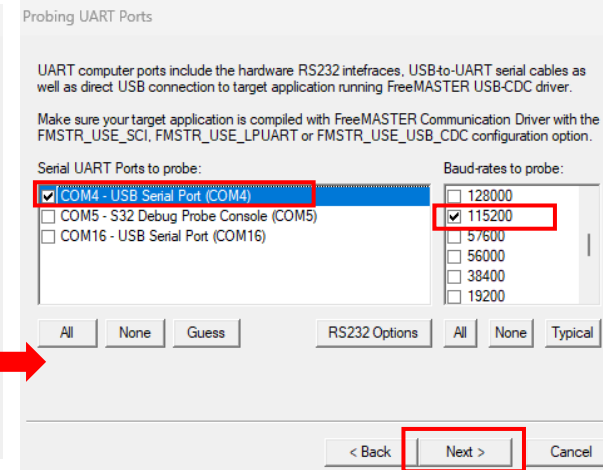
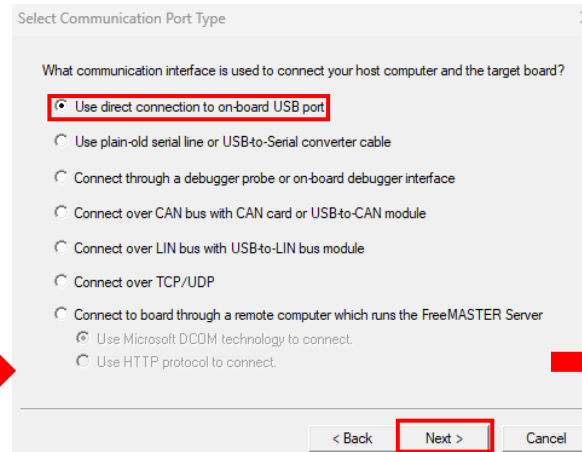
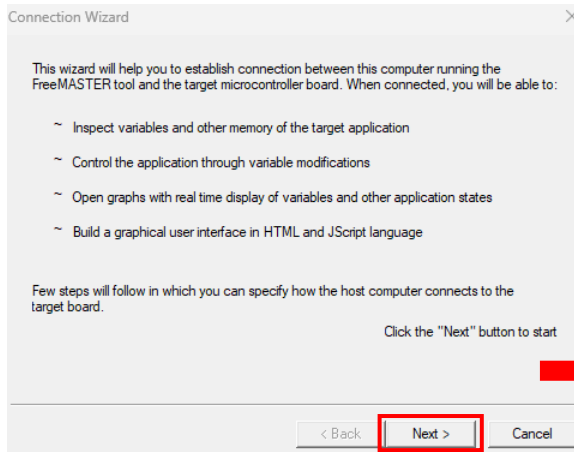
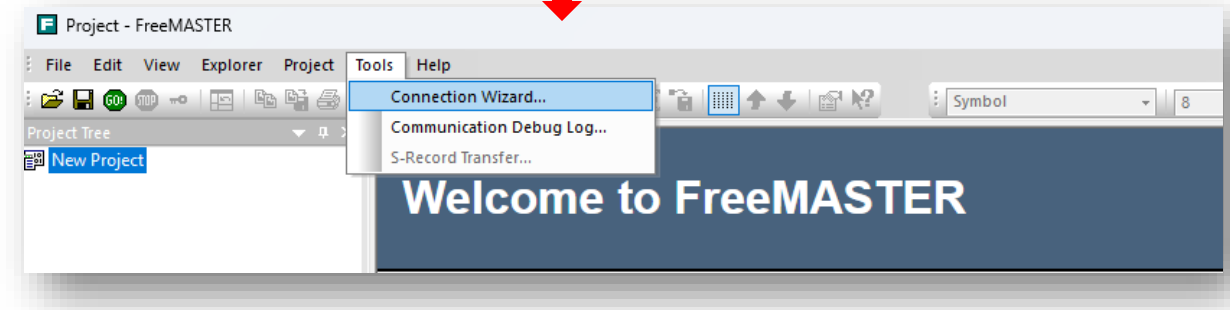
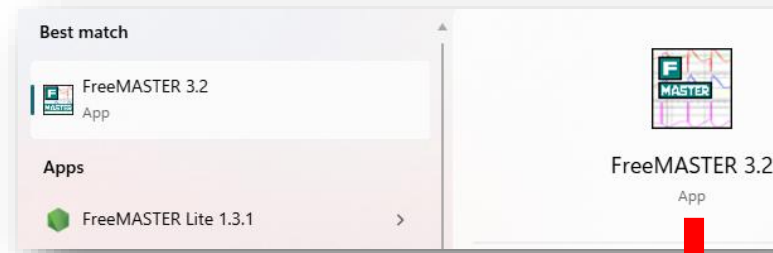




## Exercise 2: Step 5

After a successful build of the model and the elf being downloaded to the EVB, we can monitor the execution on FreeMASTER

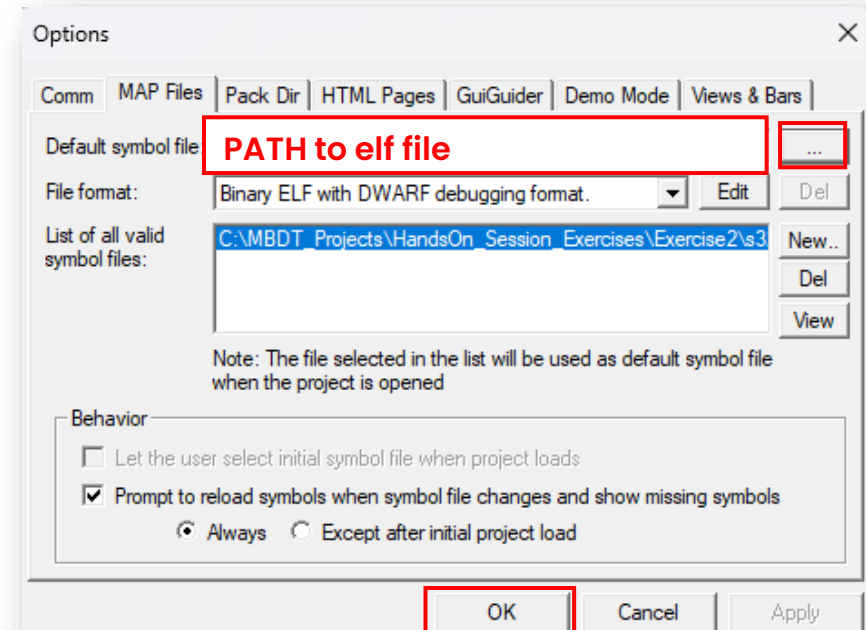
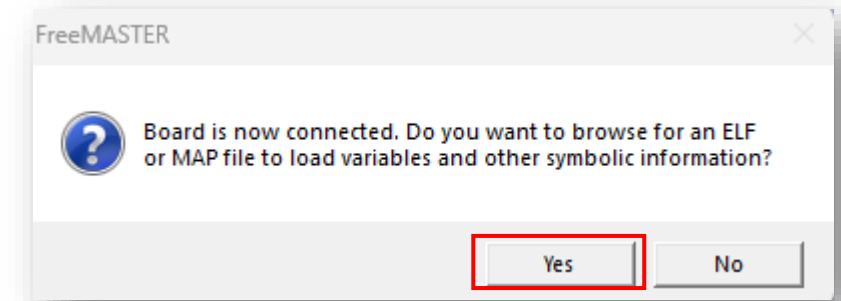
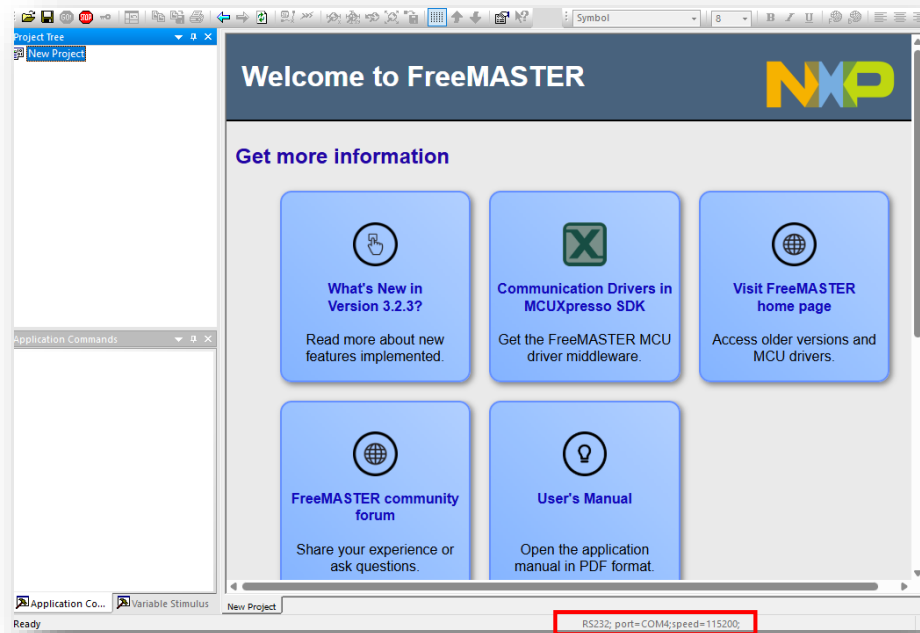
1. Open The **FreeMASTER** application
2. In **Tools** select **connection wizard**
3. Follow the step to set the COM port of the board and baudrate at 115200



## Exercise 2: Step 6

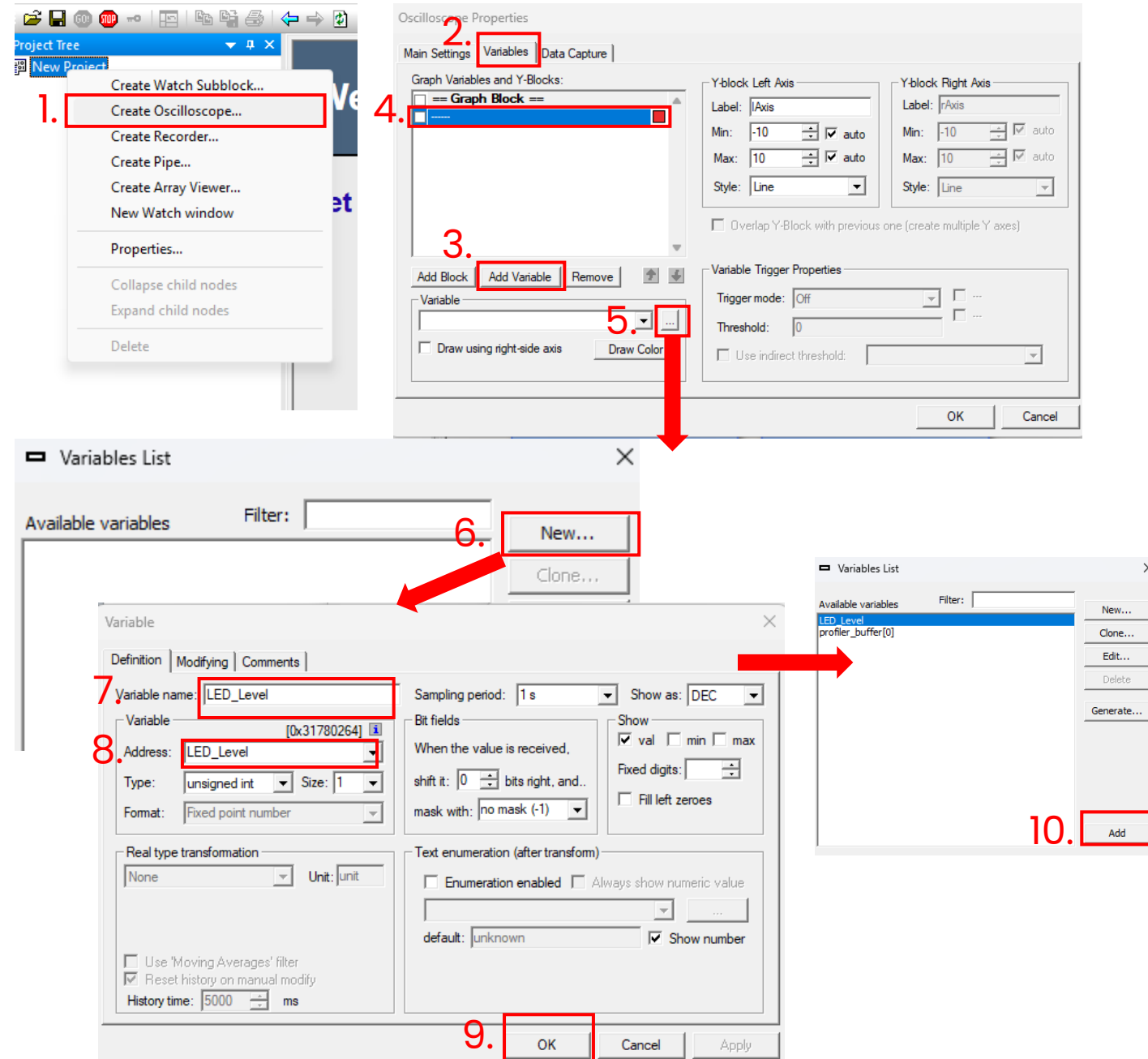
After successfully connecting the board, the elf file needs to be added to the FreeMASTER configuration

1. Click on YES on the pop up window
2. Provide path to elf file
3. Check that the board is properly connected



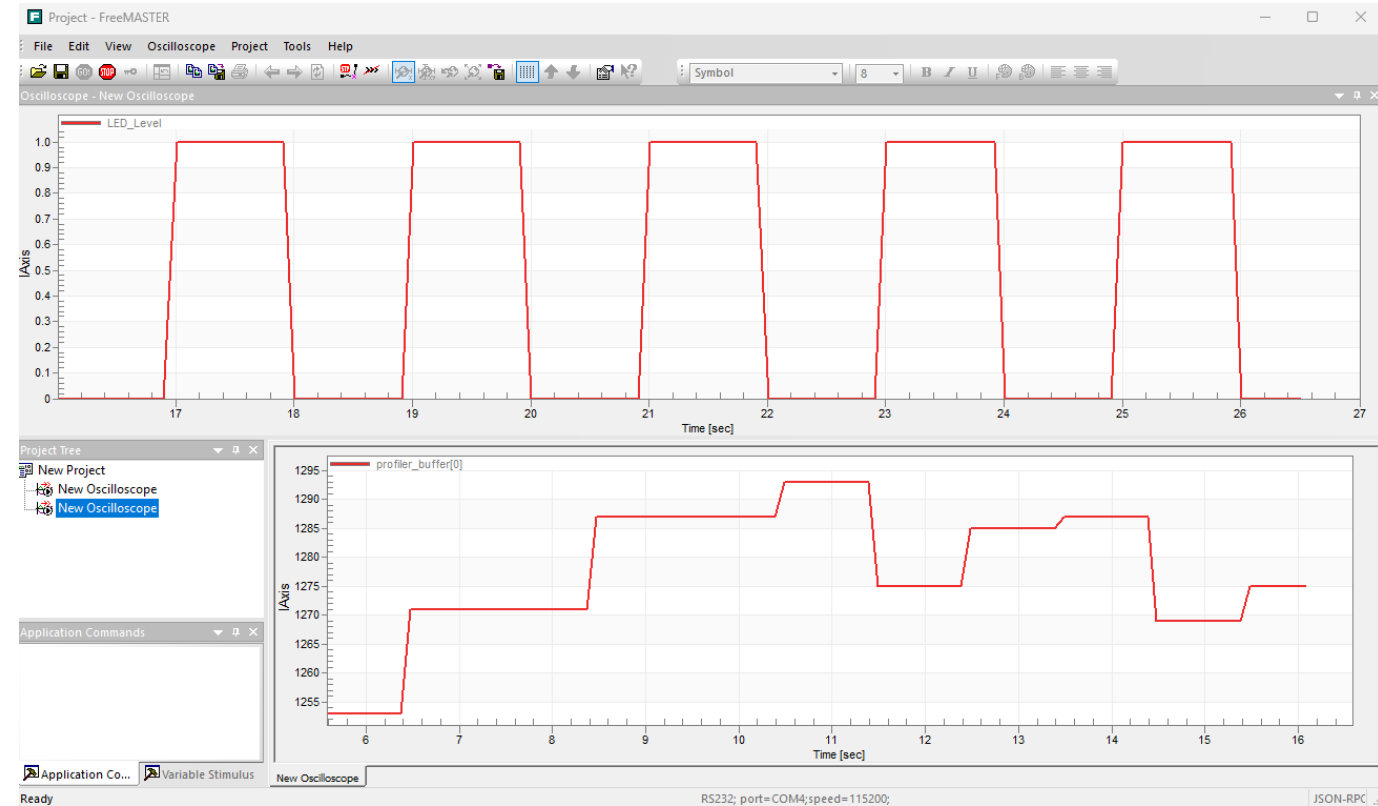
## Exercise 2: Step 7

1. Right Click on New Project then **Create Oscilloscope**
2. Select **Variables** view then click on **Add Variable**
3. Click on the 3 dots button to create add a new variable to Variables List
4. Click on New and name the new variable "LED\_Level" and in **Address** select "LED\_Level" then click OK
5. Redo the steps to add Profiler\_buffer[0] variable to the Variables list



## Exercise 2: Step 8

After successfully configuring the LED\_Level variable and the Profiling variable, the oscilloscope is available for viewing.



# Exercise 3

## Processor in the Loop



## Exercise 3: Overview

This exercise introduces **Processor-in-the-Loop (PIL)** testing using **NXP's Model-Based Design Toolbox (MBDT)** on the **S32Z2/E2 platform**. It demonstrates how to validate and profile algorithm execution directly on one of the selected cores.

### What You Will Learn

- How to configure and run a **PIL simulation** in Simulink.
- How to select and target a specific **core** on the S32Z2/E2 device.
- How to **profile execution time** and analyze performance metrics.

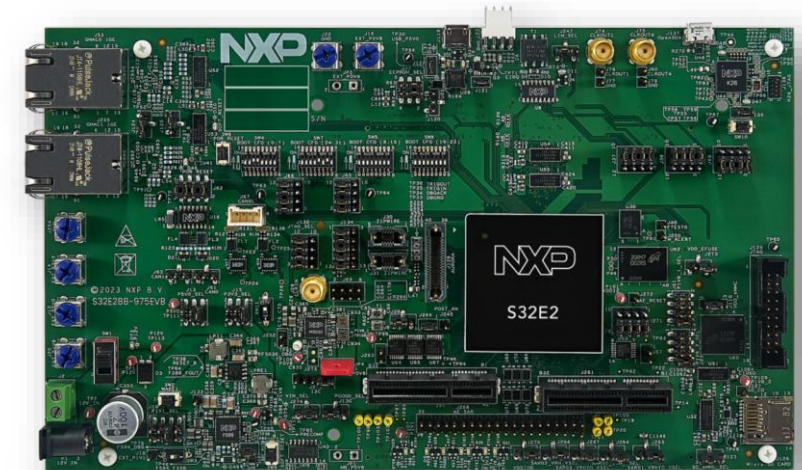
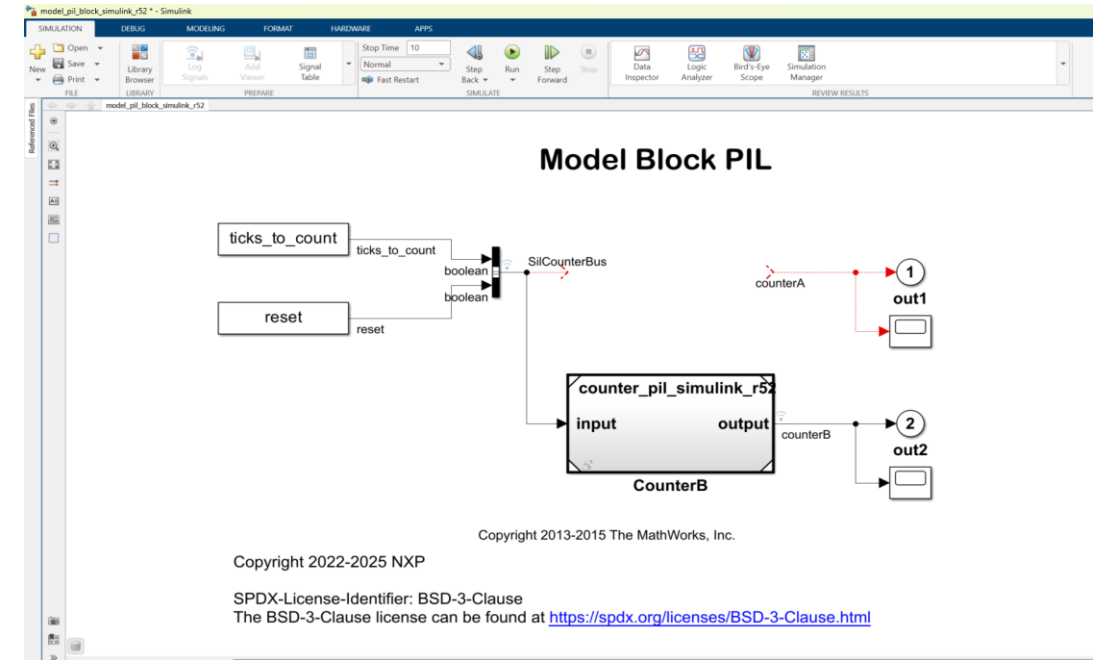
### Expected Outcome

- Understand the workflow for PIL testing using MBDT.
- Successfully execute and validate an algorithm on a selected core.
- Collect and interpret **execution profiling data** to assess performance.

## Exercise 3: Overview

For this exercise, we will start with a partially completed Simulink model and configure it to achieve the following:

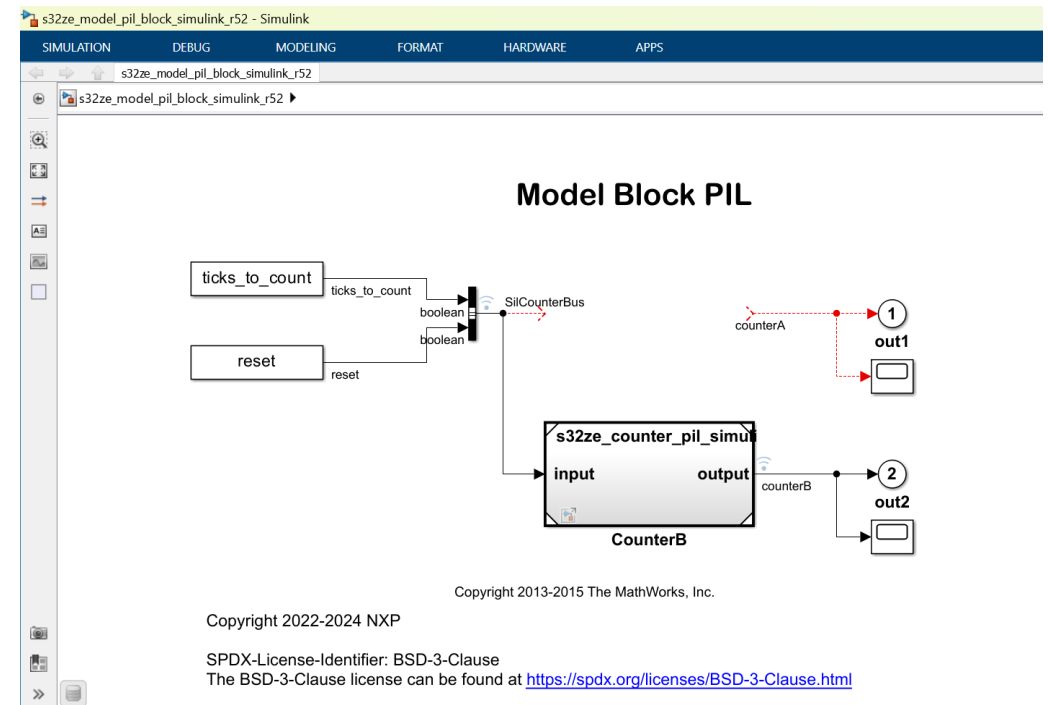
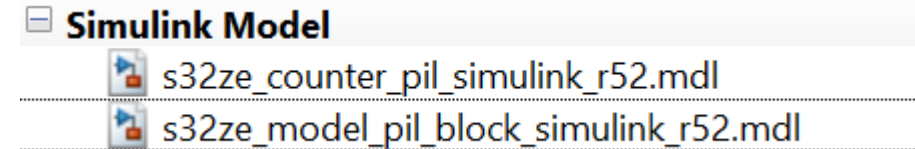
- Build and generate code for the **S32E288-bga975 processor**
- Use a model block to reference another Simulink model
- Configure project to perform Code Profiling
- Deploy the project onto the **S32E288-975EVB**
- Compare simulation data to PIL data



## Exercise 3: Step 1

The hands-on session exercises folder contains all the models used for this sessions with the solution models for reference

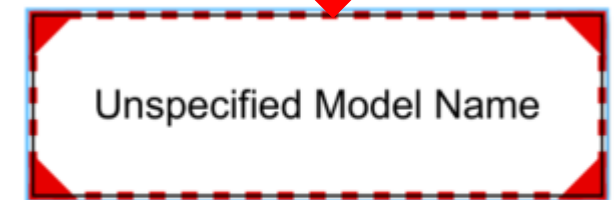
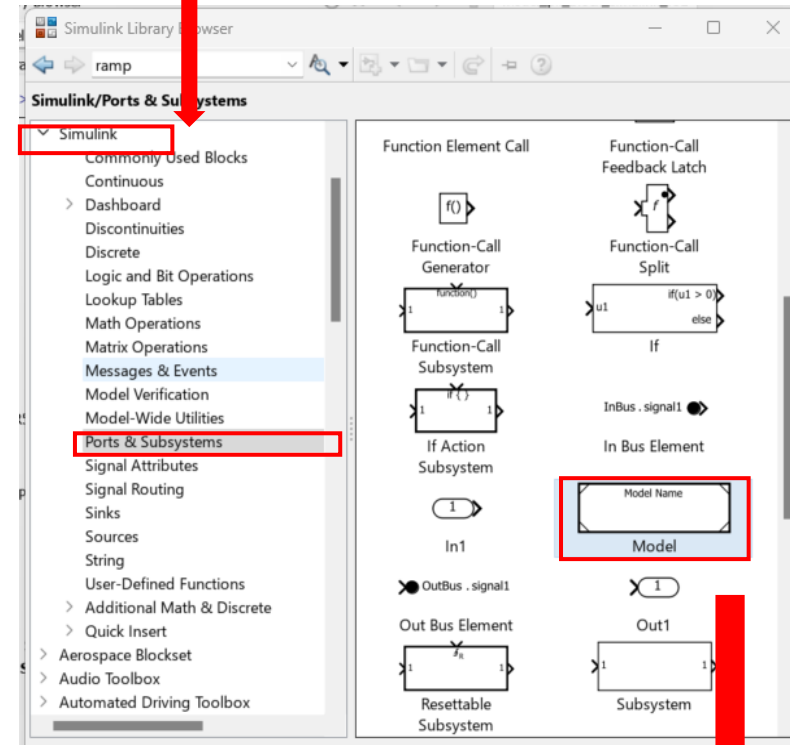
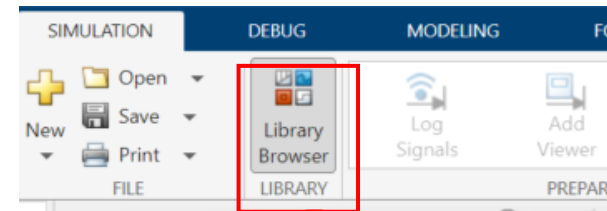
1. Open the **s32ze\_model\_pil\_block\_simulink\_r52.slx** model from **Exercise3** folder
2. The partially completed PIL Simulink model provides us with a model for running the simulation counter (B)
3. We will add Counter A that will be ran on the R52 core using PIL





## Exercise 3: Step 2

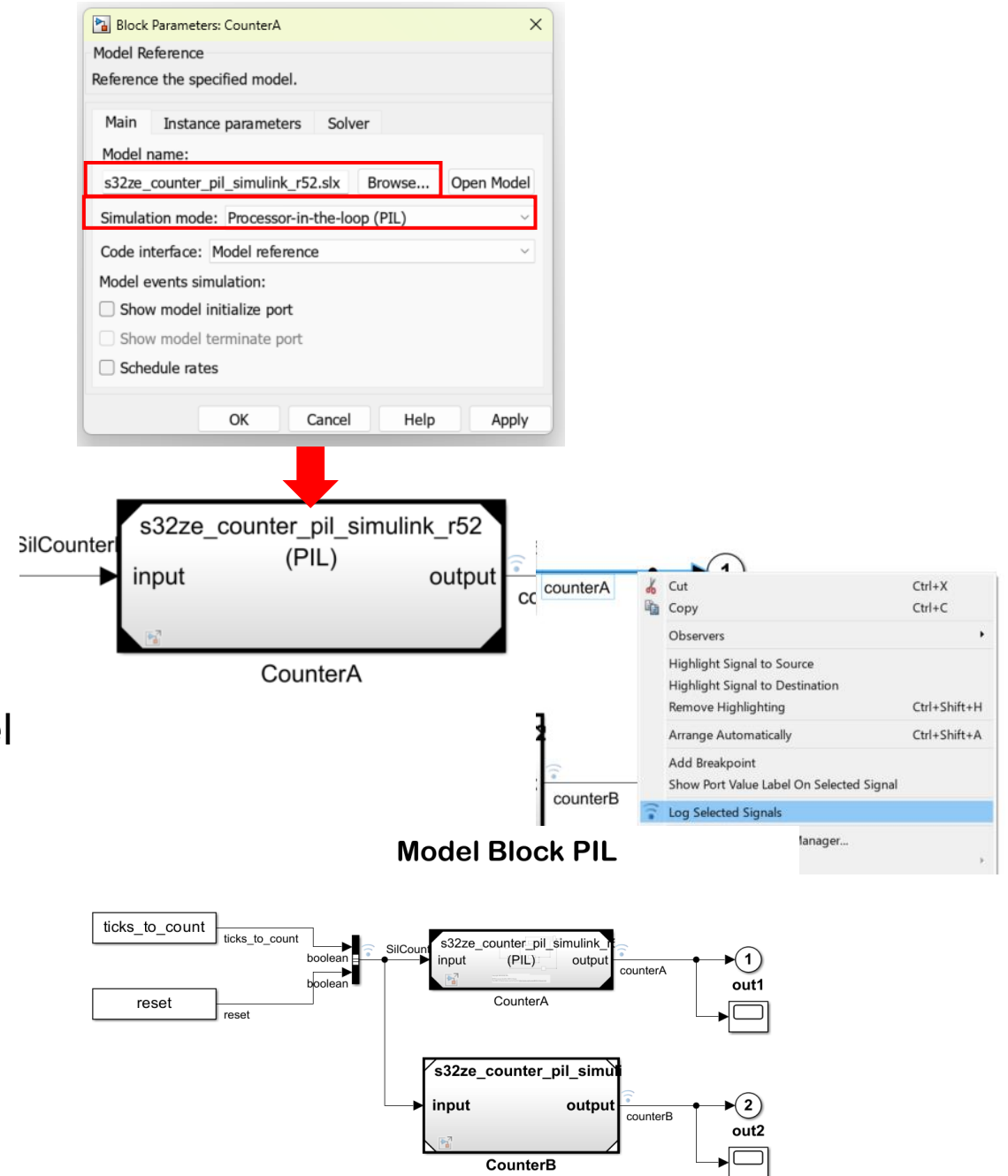
1. From the SIMULATION tab, open the **Library Browser**
2. Open the **Simulink** library -> open **Ports & Subsystems** library
3. From the **Ports & Subsystems** library, drag and drop the Model block into the Model Block PIL Simulink model
4. Name this block CounterA



Model1

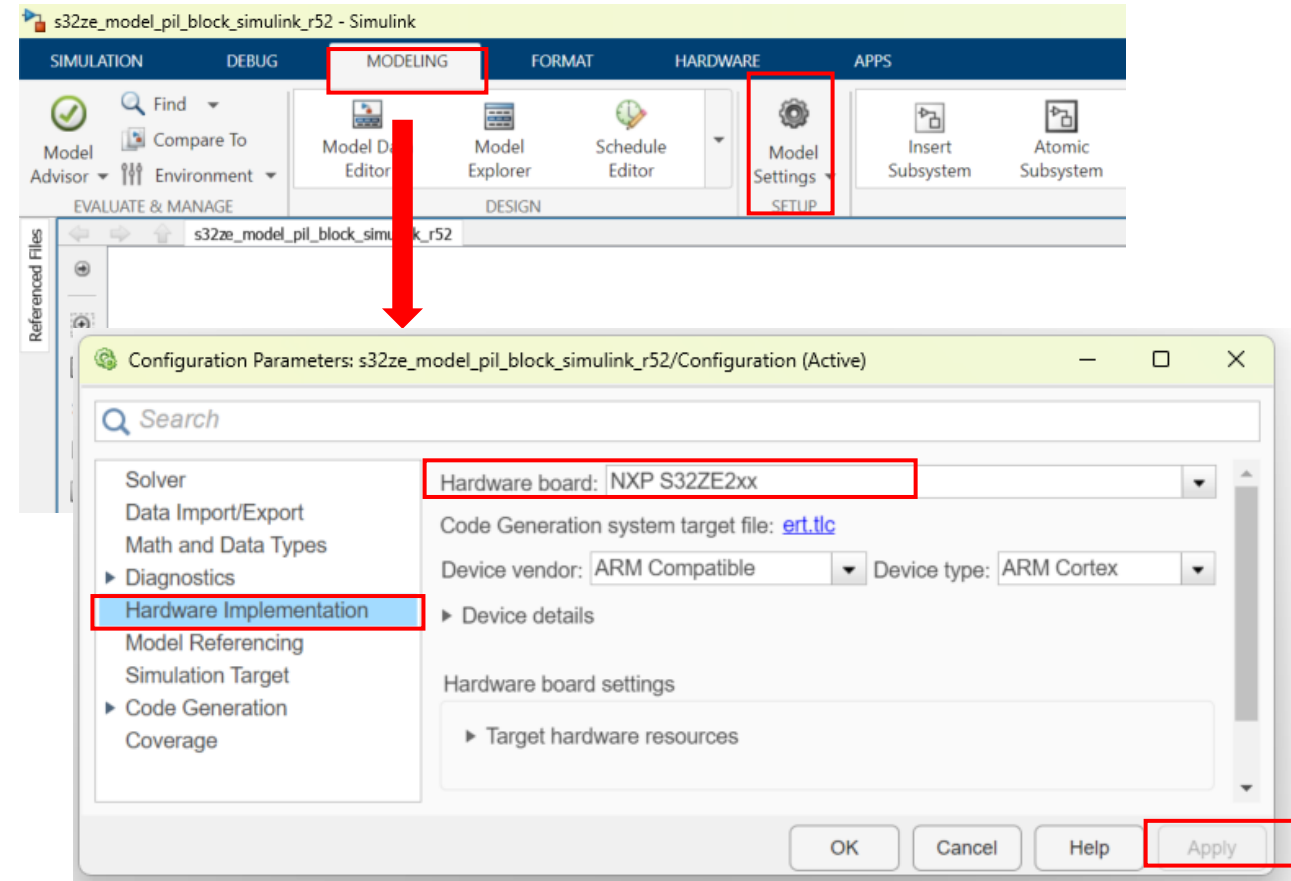
## Exercise 3: Step 3

1. Double click on the Model **block** in the Simulink model to open the **Block Parameters**
2. Select Browse and in the Select a Model window, select **s32ze\_counter\_pil\_simulink\_r52.slx** in the Exercise3 folder
3. For Simulation mode, select Processor-in-the-loop (PIL)
4. Connect **SilCounterBus** to the **input** of the model and **CounterA** to the **output**
5. Right Click the arrow that says counterA and select Log Selected Signals in the list



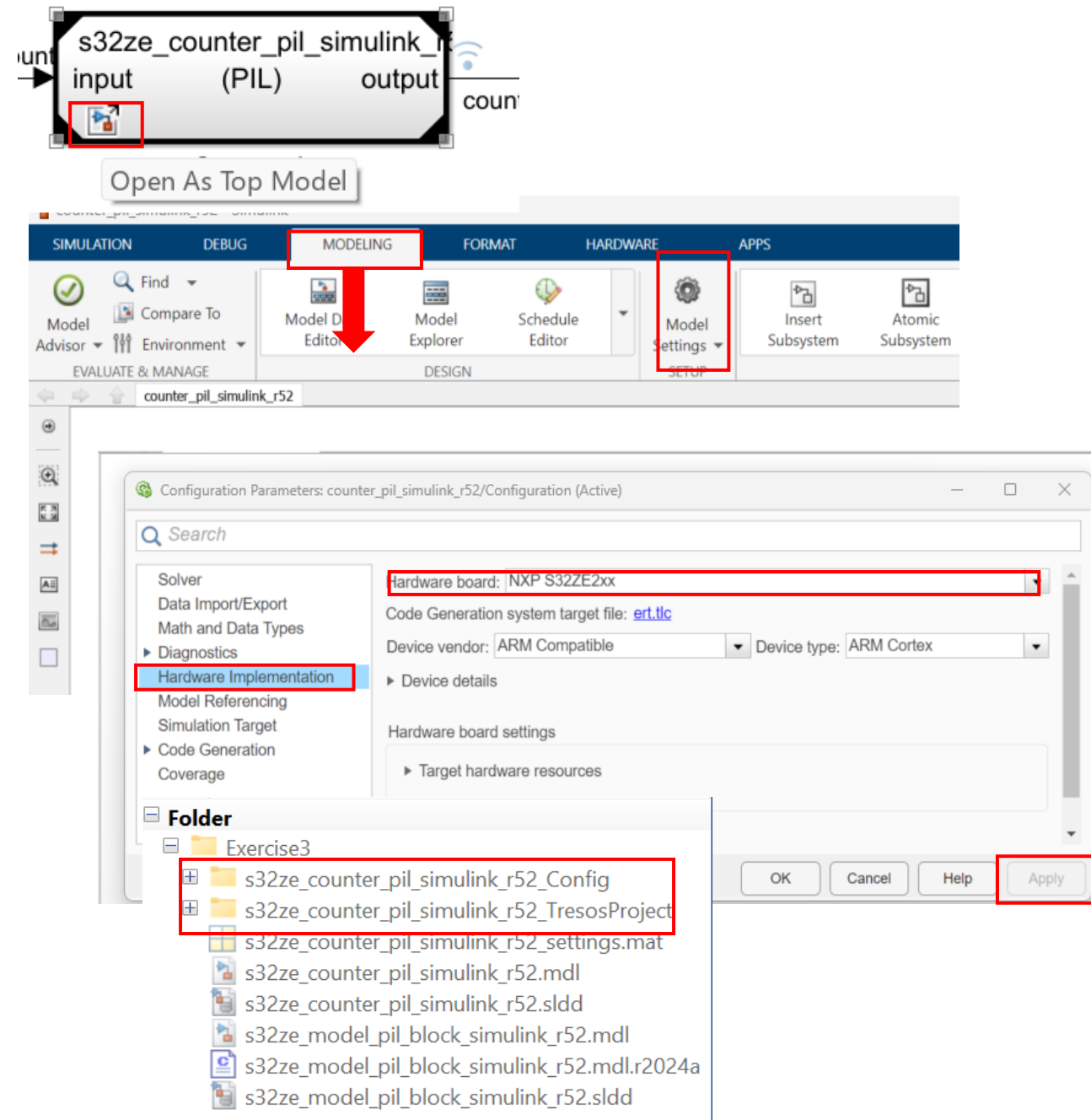
## Exercise 3: Step 4

1. In the Model Block Pil model, from the **MODELING** tab, open the **Model Settings**
2. In the **Hardware Implementation** tab of the configuration parameters window, ensure **NXP S32ZE2xx** is selected as the hardware board



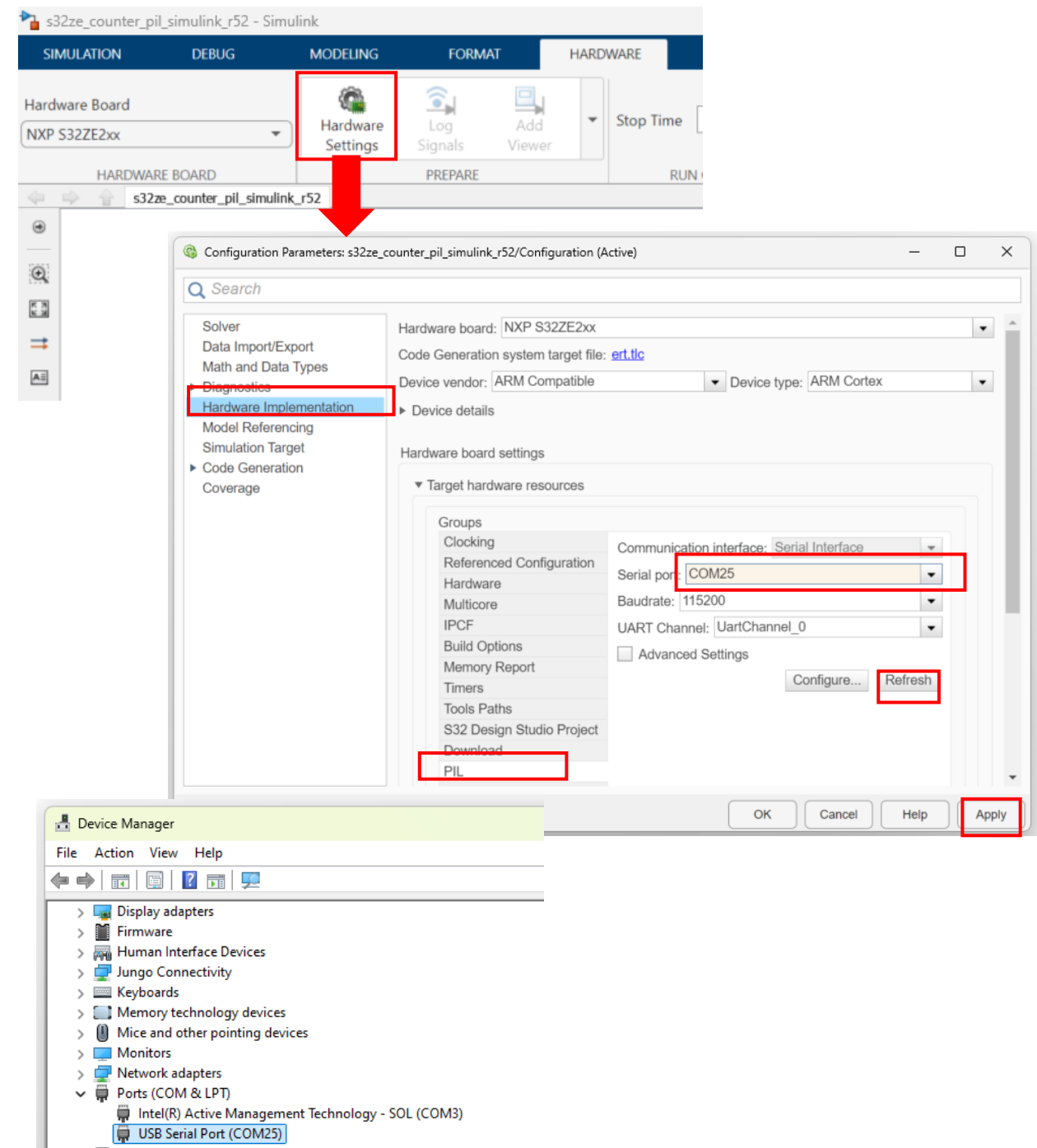
## Exercise 3: Step 5

1. Open the Counter Pil Simulink Model as the top model
2. In the Counter Pil Simulink model, from the MODELING tab, open the **Model Settings**
3. In the **Hardware Implementation** tab of the configuration parameters window, select the **NXP S32ZE2xx** as the hardware board and click on **Apply**
4. A successful configuration of the board will result in auto generating a default **S32 Design Studio** and **EB Tresos** projects for this Simulink model



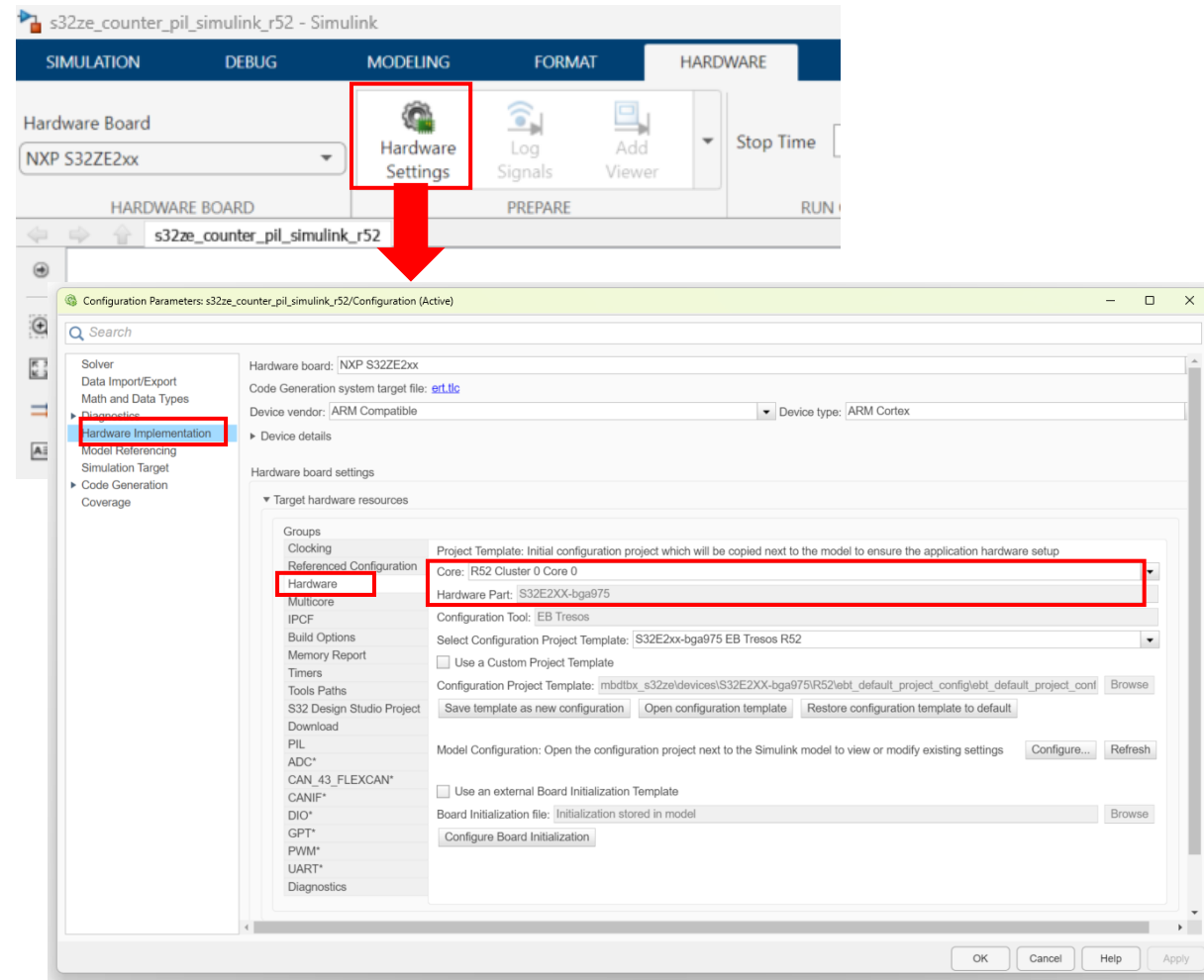
## Exercise 3: Step 6

1. In the Counter Pil Simulink model, from the **HARDWARE** tab, open the **Hardware Settings**
2. In the **Hardware Implementation** tab of the configuration parameters window, select **Target hardware resources**, then select **PIL**
3. Open Device Manager and determine which USB is connected to the board
4. Click the **Refresh** button and use the drop down for **Serial port** to select the port your board is connected to



## Exercise 3: Step 7

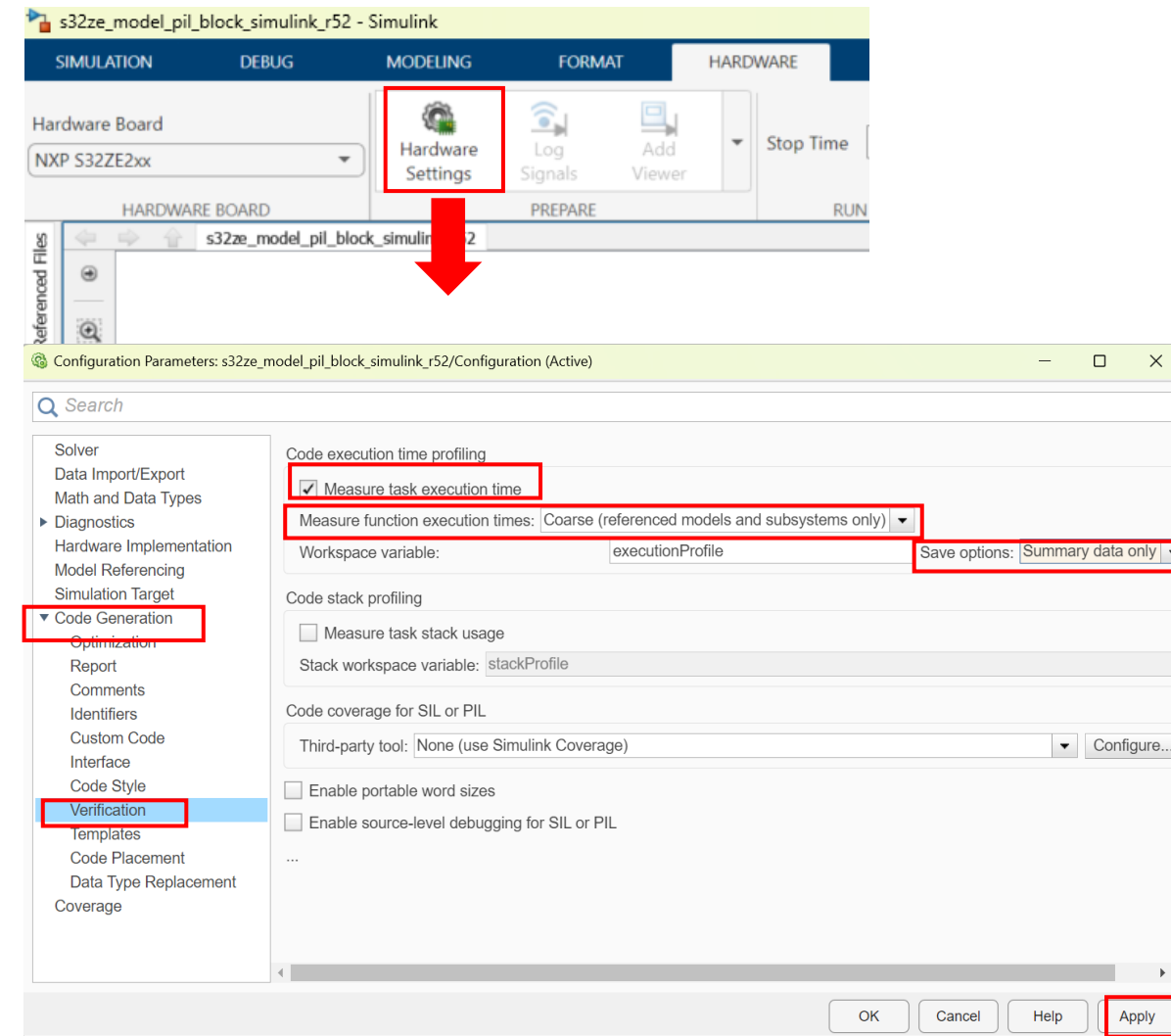
1. In the Counter Pil Simulink model, from the **HARDWARE** tab, open the **Hardware Settings**
2. Within the **Hardware Implementation** tab, expand the **Target hardware resource**, then select **Hardware**
3. Ensure the selected core is "**R52 Cluster 0 Core 0**"
4. Ensure the Hardware Part is "**S32E2xx-bga975**"



## Exercise 3: Step 8

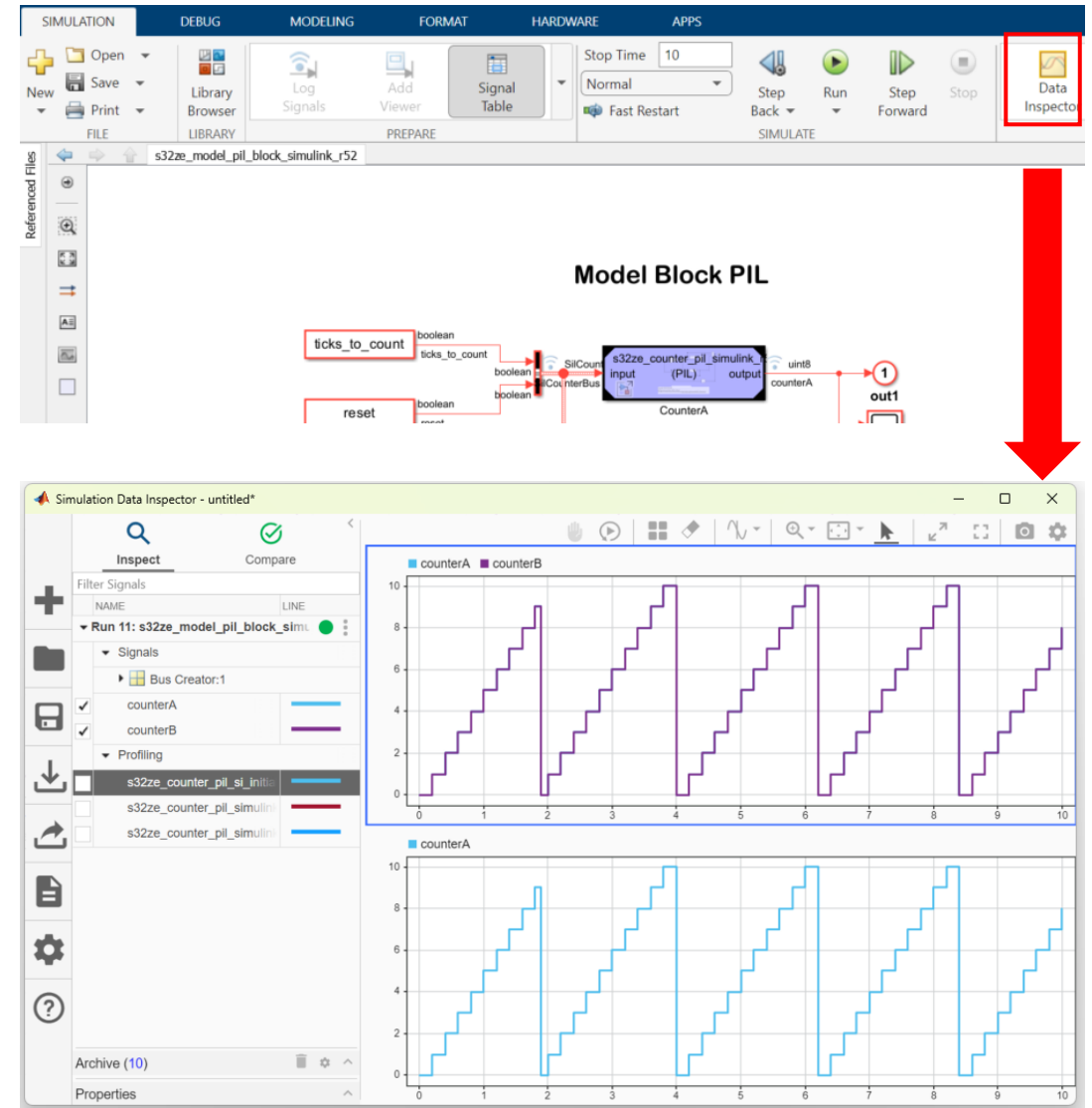
This step is for code profiling. It isn't necessary, but it's very useful for comparing PIL vs simulated data.

1. In the Model Block PIL Simulink model, from the **HARDWARE** tab, open the **Code Generation** drop-down, expand **Verification**
2. Checkmark **Measure task execution time**
3. For **Measure task execution times**, select Coarse (referenced models and subsystems only)
4. For **Save options**, select **Summary data only**



## Exercise 3: Step 9

1. Save the Model Block PIL and Counter PIL models and ensure that MATLAB's **current folder** is Exercise3
2. Connect the S32 Debugger through JTAG to the **S32E288-975EVB**, connect the microUSB cable to J119, and power up the board
3. In the Model Block PIL model, go to the **Simulation** tab and click Run
4. A successful build and deploy will result in:
  1. CounterA model block highlighted blue
  2. Clicking on **Data Inspector** should load 2 graphs where you can plot CounterA and CounterB's signals





## Exercise 3: Step 10

This step will highlight the Code Profile Analyzer tool Simulink provides to analyze performance, utilization rate, etc.

1. In the Model Block PIL model, go to the **Apps** tab, search SIL, select **SIL/PIL Manager**
2. Under the **SIL/PIL** tab, select **Compare Runs**, then in this drop-down select **Code Profile Analyzer**
3. In this window, Task Execution, Task Validation, Function Execution can be selected for inspection

The screenshot shows the Simulink interface for the 's32ze\_model\_pil\_block\_simulink\_r52' model. The 'APPS' tab is selected in the top menu. A search bar in the 'SIL' section of the 'CODE VERIFICATION, VALIDATION, AND TEST' pane shows 'SIL'. The 'SIL/PIL Manager' icon is highlighted. A red arrow points to the 'Compare Runs' button in the 'SIL/PIL' tab. Another red arrow points to the 'Code Profile Analyzer' option in the dropdown menu. Below this, the 'Code Profile Analyzer' window is shown with the 'TIME PROFILING' tab selected. The 'Task Execution' sub-tab is active, displaying a table of task execution times.

| Section Name                    | Maximum Execution Time | Average Execution Time | Maximum Self Time | Average Self Time | Calls |
|---------------------------------|------------------------|------------------------|-------------------|-------------------|-------|
| s32ze_counter_pil_si_initialize | 2.2000e-04             | 2.2000e-04             | 2.2000e-04        | 2.2000e-04        | 1     |
| s32ze_counter_pil_simulink_...  | 2.2250e-04             | 2.2250e-04             | 2.2250e-04        | 2.2250e-04        | 1     |
| s32ze_counter_pil_simulink_...  | 4.0500e-04             | 3.3322e-04             | 4.0500e-04        | 3.3322e-04        | 101   |

The right sidebar of the 'Code Profile Analyzer' window shows the 'Summary' tab with the following information:

- Profiled component: s32ze\_model\_pil\_bloc...
- Profiled models: s32ze\_counter\_pil\_sim...
- Workflow: SIL/PIL
- Environment: Simulink
- Save options: Summary data only
- Time created: 24-Oct-2025 14:39:16
- Instrumentation over...: Filtered (benchmark)
- Timer driver overhead: 9e-05
- Task overhead: 9.5e-05
- Function overhead: 0.000225
- Timer freezing overh...: 9e-05

# Exercise 4

## Multicore Execution



## Exercise 4: Overview

This exercise introduces **Multicore** processing using **NXP's Model-Based Design Toolbox (MBDT)** on the **S32Z2/E2 platform**. It demonstrates how to use **Simulink Concurrent Execution** to run a multicore application on ARM Cortex-R52 cores.

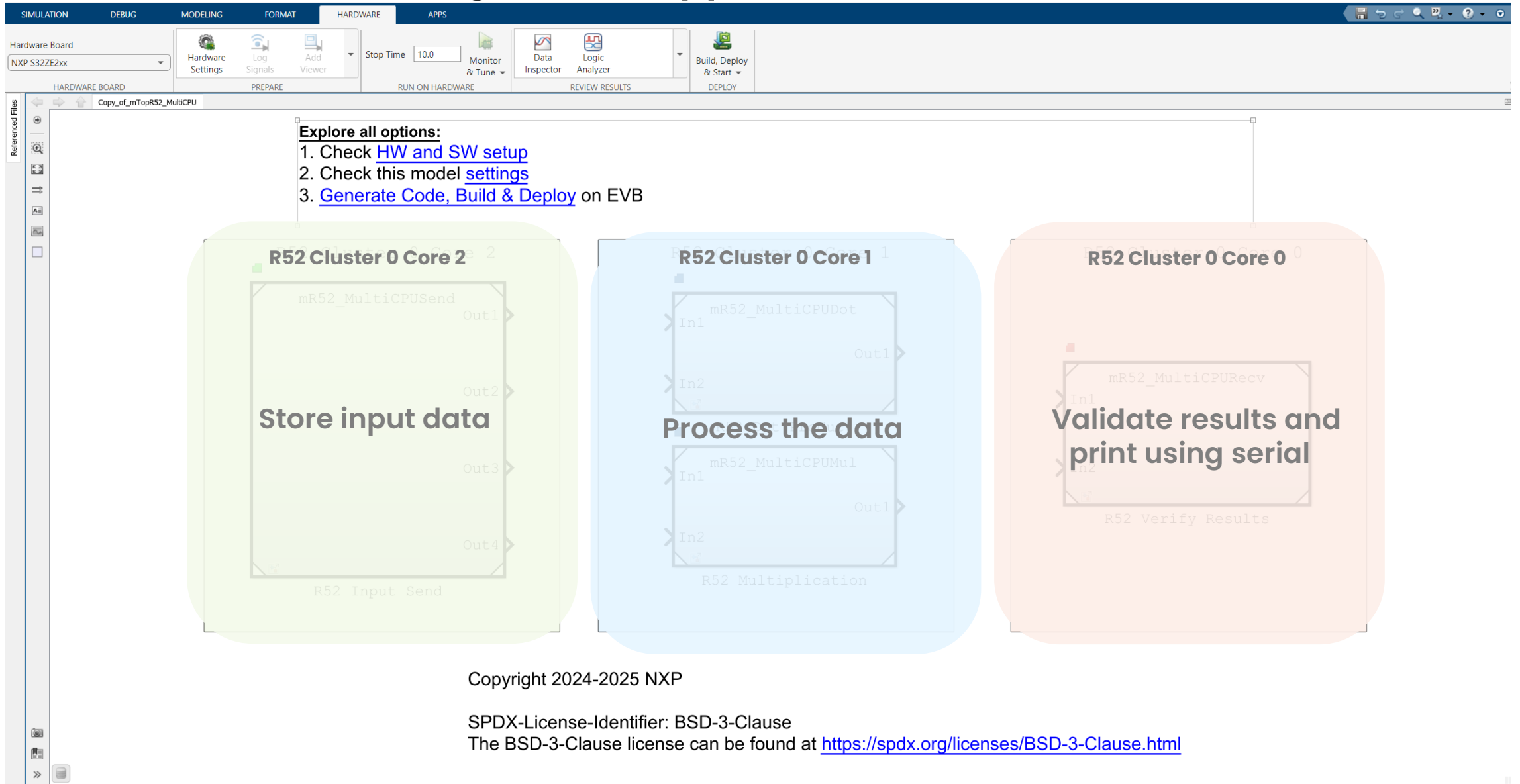
### What You Will Learn

- How to configure and run a multicore example in Simulink.
- How to map Simulink models to a specific **core** on the S32Z2/E2 device.

### Expected Outcome

- Understand the workflow for multicore processing using MBDT.
- Successfully build and deploy a multicore Simulink model to the EVB.

# Exercise 4: Overview – Single-Core Applications

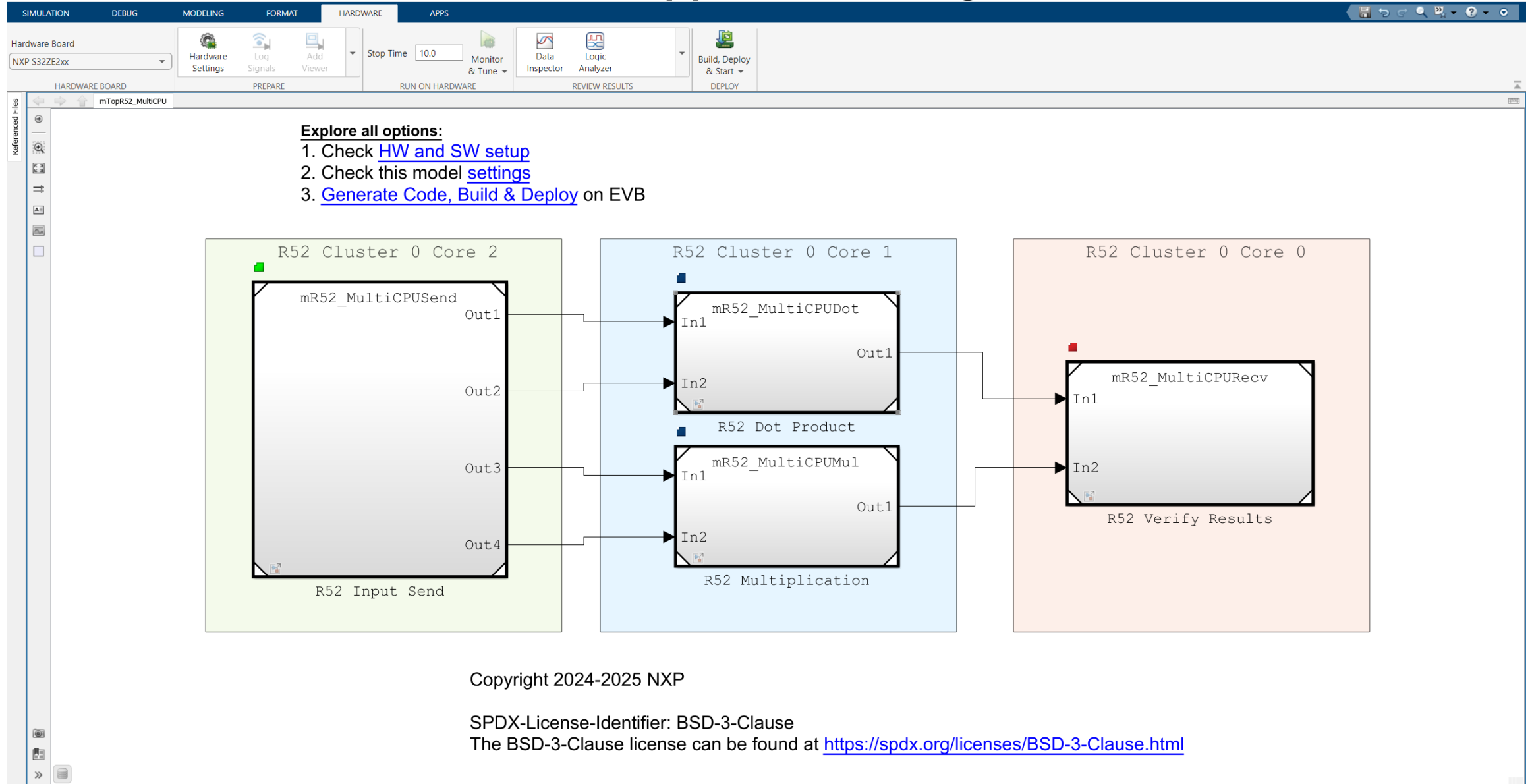


Copyright 2024-2025 NXP

SPDX-License-Identifier: BSD-3-Clause

The BSD-3-Clause license can be found at <https://spdx.org/licenses/BSD-3-Clause.html>

# Exercise 4: Overview – Multi-Core Application using Simulink Concurrent Execution



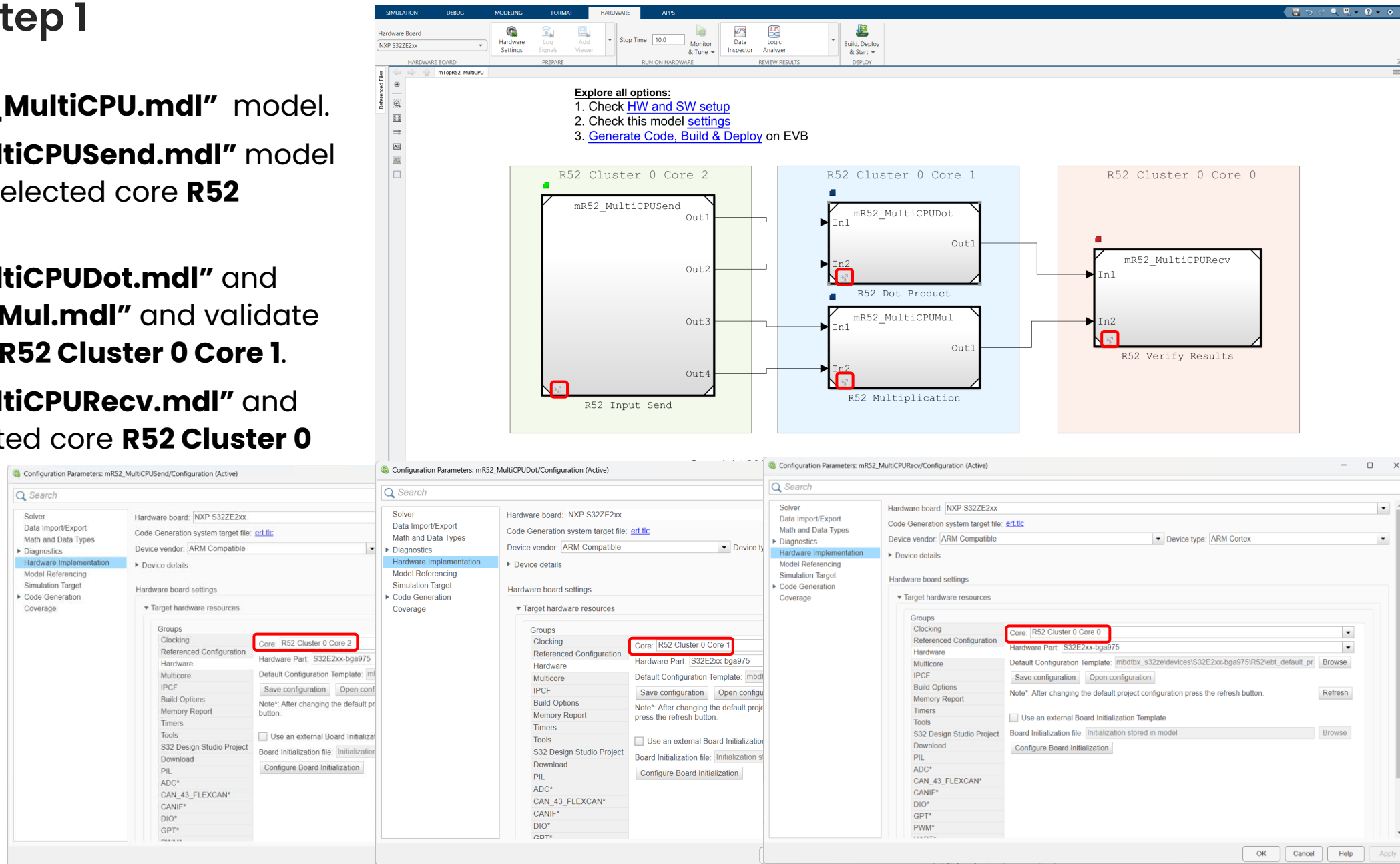
Copyright 2024-2025 NXP

SPDX-License-Identifier: BSD-3-Clause

The BSD-3-Clause license can be found at <https://spdx.org/licenses/BSD-3-Clause.html>

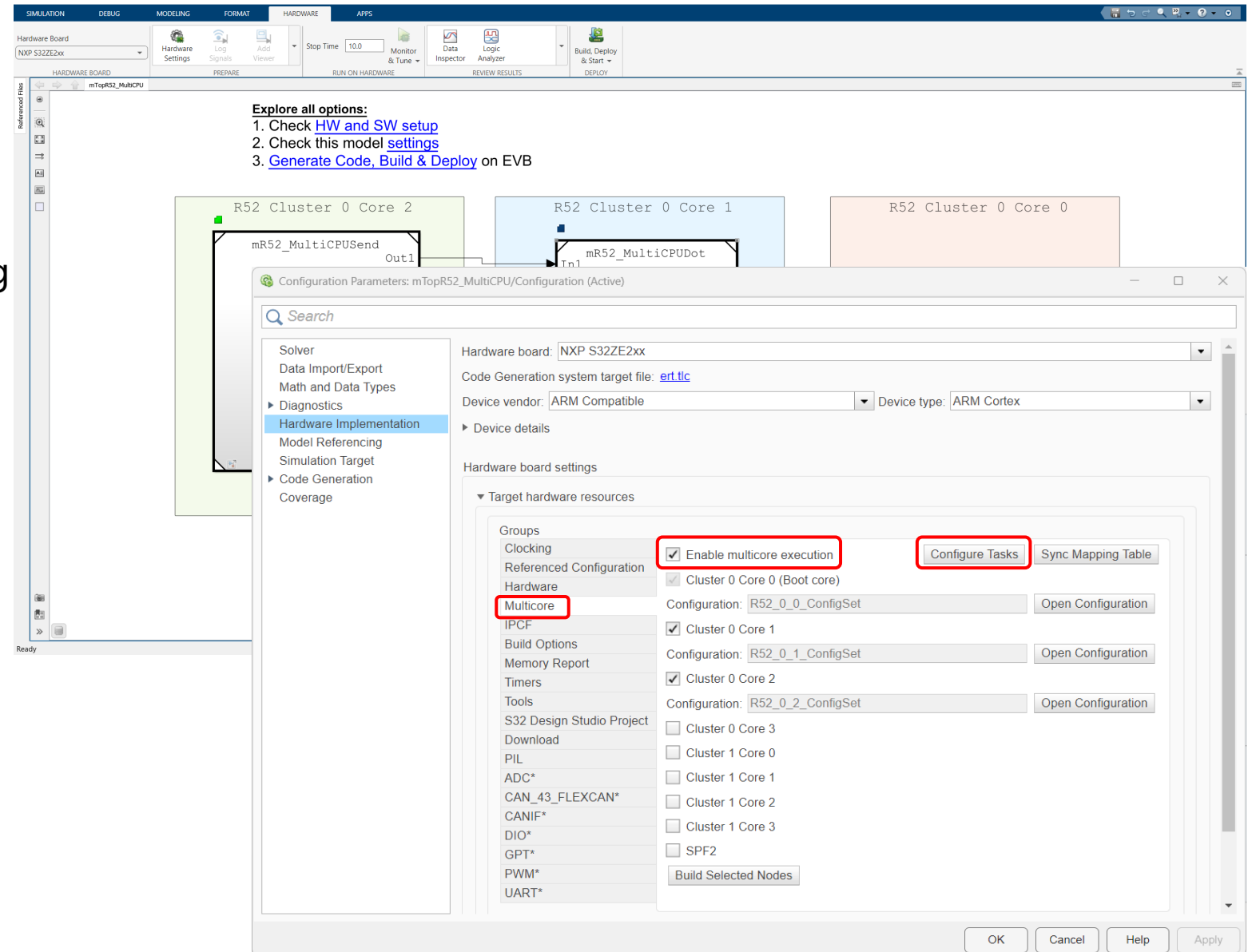
# Exercise 4: Step 1

- Open “mTopR52\_MultiCPU.mdl” model.
- Open “mR52\_MultiCPUSend.mdl” model and validate the selected core **R52 Cluster 0 Core 2**.
- Open “mR52\_MultiCPUDot.mdl” and “mR52\_MultiCPUMul.mdl” and validate the selected core **R52 Cluster 0 Core 1**.
- Open “mR52\_MultiCPURecv.mdl” and validate the selected core **R52 Cluster 0 Core 0**.



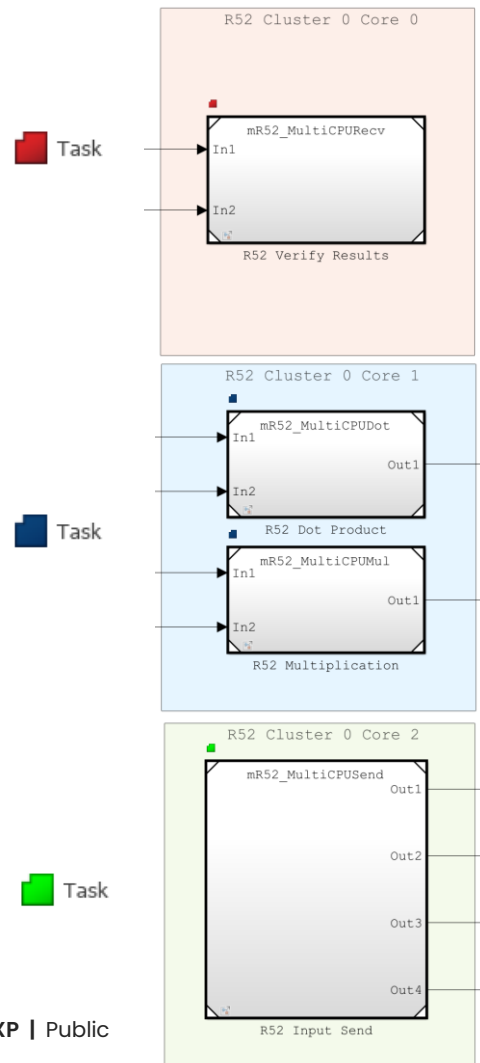
## Exercise 4: Step 2

- Open **Hardware Settings** for “**mTopR52\_MultiCPU.mdl**” top model.
- Check in **Multicore** group if ***Enable multicore execution*** is enabled.
- Open the Concurrent Execution GUI using the **Configure Tasks** button



# Exercise 4: Step 3

- Check for correct core mapping in the **Concurrent Execution GUI**.



Explore all options:  
1. Check [HW and SW setup](#)  
2. Check this model [settings](#)  
3. [Generate Code, Build & Deploy](#) on EVB

Concurrent Execution: mTopR52\_MultiCPU (Active)

CONCURRENT EXECUTION

TASK TRIGGER MAPPING DELETE

Map blocks to tasks

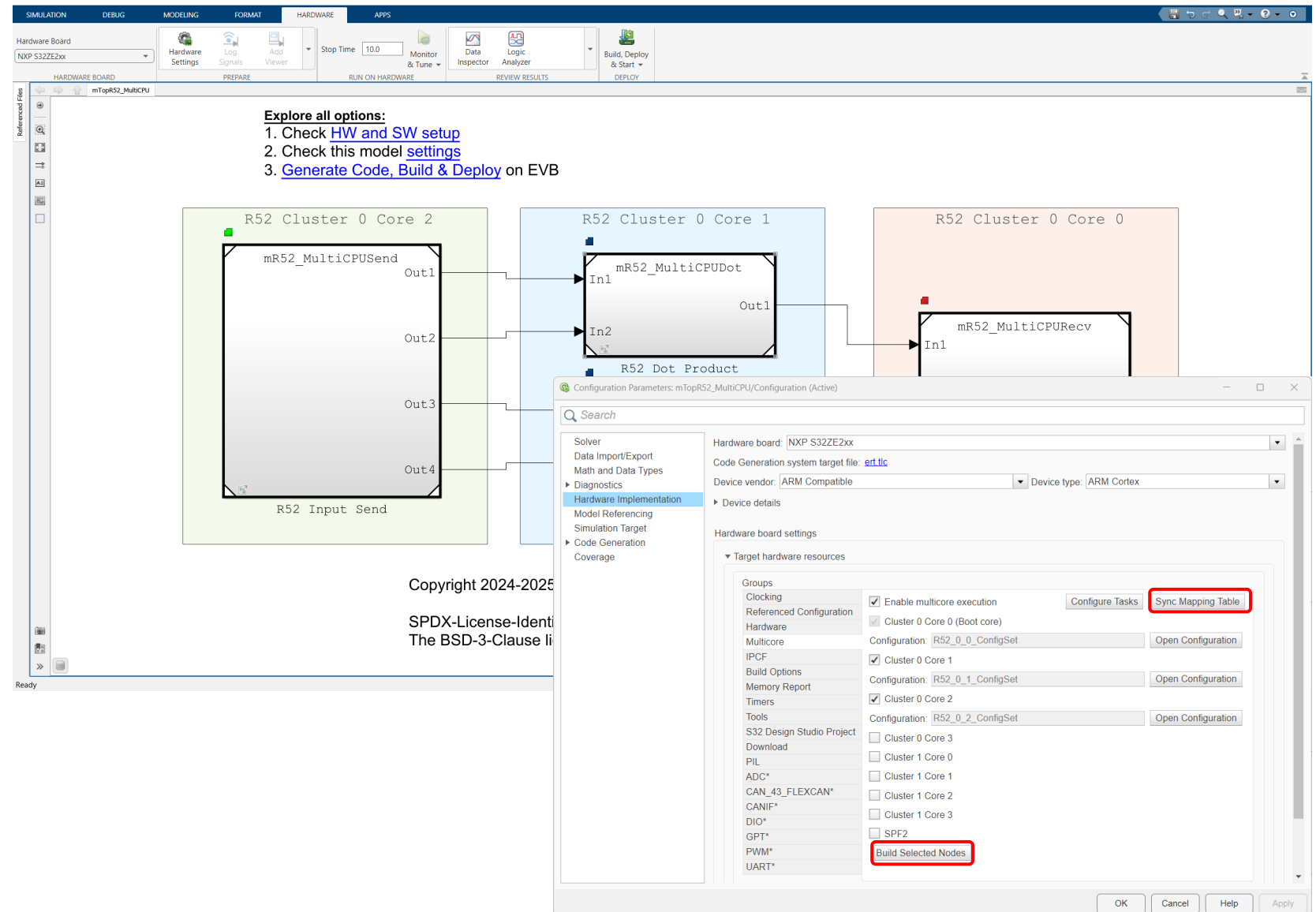
Refresh mapping table (\*invokes update diagram)

| Block                     | Name                               | TriggerType | Period | Autogenerated |
|---------------------------|------------------------------------|-------------|--------|---------------|
| Block: R52 Dot Product    |                                    |             |        |               |
| ■ R52 Dot Product         | R52_CLUSTER_0_CORE_1:Periodic:Task | Periodic    | 0.2    | No            |
| Block: R52 Input Send     |                                    |             |        |               |
| ■ R52 Input Send          | R52_CLUSTER_0_CORE_2:Periodic:Task | Periodic    | 0.2    | No            |
| Block: R52 Multiplication |                                    |             |        |               |
| ■ R52 Multiplication      | R52_CLUSTER_0_CORE_1:Periodic:Task | Periodic    | 0.2    | No            |
| Block: R52 Verify Results |                                    |             |        |               |
| ■ R52 Verify Results      | R52_CLUSTER_0_CORE_0:Periodic:Task | Periodic    | 0.2    | No            |



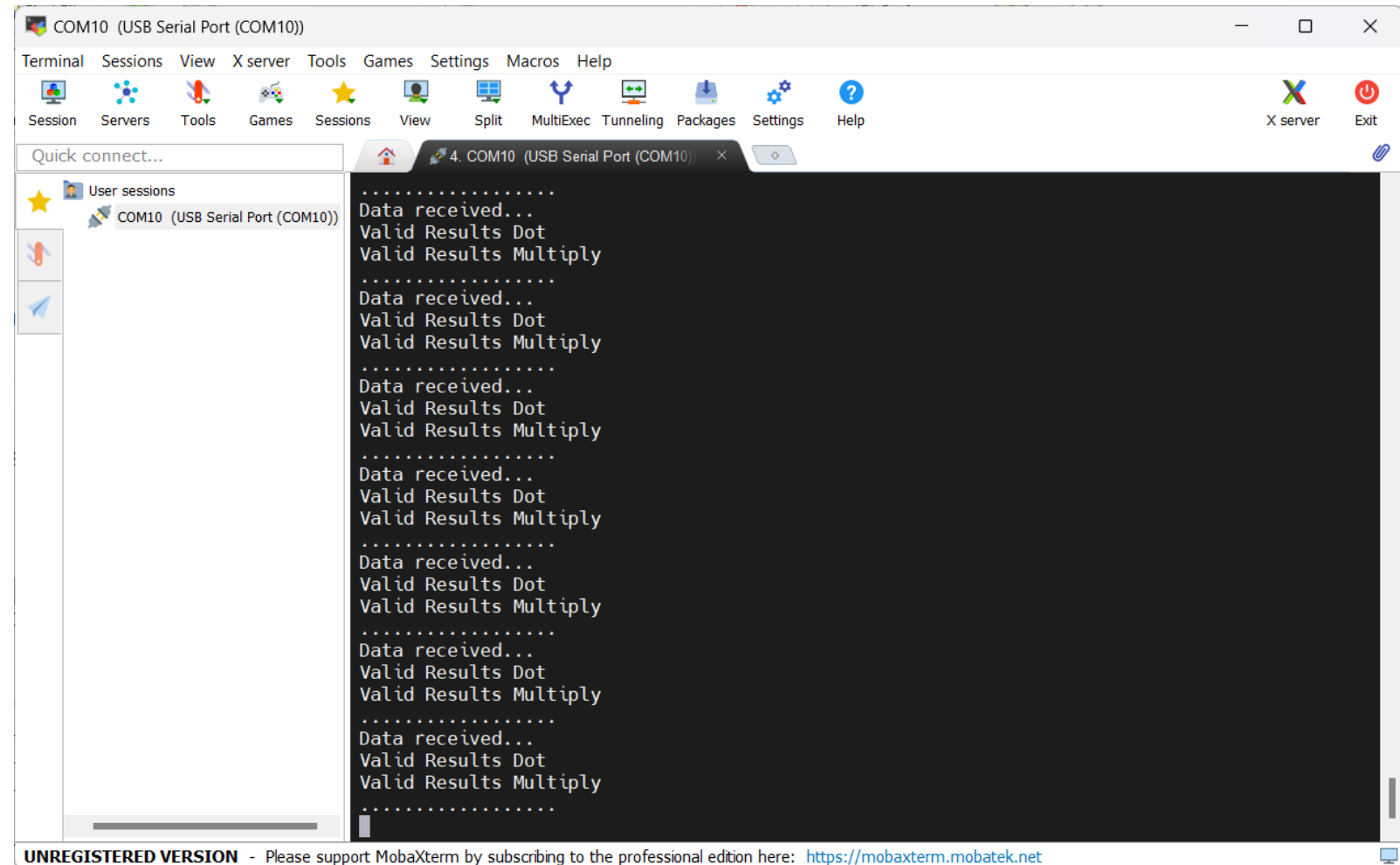
## Exercise 4 : Step 4

- Refresh mapping table to sync in case of any modifications
- Connect the S32 Debugger through JTAG to the **S32E288-975EVB**, connect the microUSB cable, and power up the board
- Build and deploy the multicore example using the **Build Selected Nodes** button



## Exercise 4 : Step 5

- Open the serial terminal to visualize the results



# Additional Resources and Support

NXP: ソフトウェア開発工数の増大でお悩みの方必見。  
NXPのMBDTソリューション

NXP: Deploying Radar Applications to S32R41

MW: Hardware-Aware Simulink Applications using  
MBDT

NXP: Deploying BMS Applications to S32K3xx and  
HVBMS Reference Design

NXP: Motor Control: PMSM S32K3xx

MW: Motor Control: S32K1xx

MW: Motor Control: i.MX RT

NXP: Motor Control: BLDC/PMSM

NXP: Motor Control: Design Application

MW: Code Generation and Verification

MW: Speed Up Applications Development with MBDT

MW: Rapid Prototyping of Embedded Designs with  
MBDT

MW: AUTOSAR SW on S32K1xx/MPC57xx

NXP: AUTOSAR SW on S32K3xx

MW: Deploying BMS Algorithm to S32K1xx

MW: Deploying Deep Learning SOC algorithm on  
S32K3xx

MW: HIL Testing of Balance of Plant Controller of Fuel  
Cell System

MW: Vision Applications for S32V

NXP: Machine Vision Algorithm Development

NXP: FreeMASTER

MW: Automotive DevOps for MBD with AWS

MW: Integrating AI-Based Virtual Sensors into Model-  
Based Design

Home / Model-Based Design Toolbox (MBDT) / Model-Based Design Toolbox (MBDT)

## Model-Based Design Toolbox (MBDT)

All community

Search all content

### MBDT Beginner's Guide

W1: MBDT Introduction

W2: How-To SPI

Options W3: How-To CAN

W4: How-To PWM

W5: How-To LIN

W6: How-To PIL

W7: How-To Timers

### Model-Based Design Tools for MATLAB and Simulink Support

#### S32K1xx



- How to
- Tutorials
- Videos
- FAQ

#### S32K3xx



- How to
- Tutorials
- Videos
- FAQ

#### BMS



- How to
- Tutorials
- Videos
- FAQ

#### S32M2xx



- How to
- Tutorials
- Videos
- FAQ

#### S32ZE



- How to
- Tutorials
- Videos
- FAQ

#### HCP



- How to
- Tutorials
- Videos
- FAQ

#### MPC57xx



- How to
- Tutorials
- Videos
- FAQ

#### S12ZVM



- How to
- Tutorials
- Videos
- FAQ

#### i.MX RT



- How to
- Tutorials
- Videos
- FAQ

#### Kinetis V



- How to
- Tutorials
- Videos
- FAQ

#### DSC



- How to
- Tutorials
- Videos
- FAQ

#### LPC 553x



- How to
- Tutorials
- Videos
- FAQ

#### LAX



- How to
- Tutorials
- Videos
- FAQ

#### Radar/S32R



- How to
- Tutorials
- Videos
- FAQ

#### Vision/S32V234



- How to
- Tutorials
- Videos
- FAQ

#### MathWorks



- NXP Support
- Motor Control
- BMS
- Videos

The Model-Based Design Toolbox provides an integrated development environment and toolchain for configuring and generating all the necessary software automatically (including initialization routines and device drivers) to execute complex applications (e.g., battery management systems, motor control algorithms, communication protocols CAN, LIN, SPI, I2C, UART, and sensor-based applications) on NXP microcontrollers and microprocessors.



### PMSM Control Workshop

Course Main Page

M1: Environment Setup

M2: PMSM and FOC

M3: System Partitioning

M4: PWM Modulation

M5: V/f Scalar Control

M6: Current Sensing

M7: Torque Control

M8: Speed Control

M9: Position Observer

M10: Sensorless Speed Control

### BLDC Control Workshop

Course Main Page

M1. Introduction

M2. Application Partitioning

M3. Input Commands

M4. BLDC Motor Theory

M5. Hall Sensors

M6. Commutation

M7. Commutation Algorithm

M8. Power Stage Config

M9. Open Loop Control

M10. Speed Estimator

M11. Closed Loop Control

M12. Motor Control System

# Appendix



# How to use MobaXterm

- Open MobaXterm application
- Click on Session
- Select Serial
- Configure the Serial port
- Configure serial speed to 115200 bps
- Click on OK

